

DEPARTMENT OF MATHEMATICS "TULLIO LEVI-CIVITA"

BACHELOR'S DEGREE IN COMPUTER SCIENCE



UNIVERSITÀ
DI PADOVA

Explainable Machine Learning through Large Language Models: Analysis and Prompt Design

Bachelor's Thesis

Supervisor

Prof. Michele Scquizzato

Undergraduate

Lorenzo Soligo

ID number 2101057

TO DO

— TO DO

Dedicato a ...

Abstract

As [Machine Learning \(ML\)](#) models become increasingly prevalent in business applications, ensuring their transparency and reliability through [Explainable Artificial Intelligence \(XAI\)](#) is critical. This project, conducted in collaboration with Zucchetti S.p.A., investigates the interpretability and comprehensibility of various machine learning algorithms.

The research systematically evaluates a spectrum of predictive models, ranging from fundamental [Regression \(regression\)](#) and [Classification \(classification\)](#) models—such as Linear Regression, Logistic Regression, Support Vector Machines, and Decision Trees—to complex ensemble methods, including Random Forest, XGBoost, RuleFit, and Symbolic Regression. The analysis explores both intrinsic model transparency and post-hoc explainability techniques, utilizing metrics like [SHapley Additive exPlanations \(SHAP\)](#) to interpret predictions and feature importance. Furthermore, the methodology encompasses the examination of mathematical foundations, internal knowledge representations, and the impact of ensemble techniques on both model performance and explainability. Real-world validation is performed using datasets such as Student Salary Prediction.

A primary objective of this work is bridging the gap between technical accuracy and human-understandable explanations for non-expert stakeholders. To achieve this, the project leverages advanced Prompt Engineering principles. Tailored prompts for [Large Language Model \(LLM\)](#) are developed for each analyzed algorithm. These prompts are explicitly designed to automatically generate human-readable explanations of algorithmic behaviors and predictions. Ultimately, the project delivers production-ready implementations and [LLM](#) integration adapters , demonstrating how the synthesis of traditional [XAI](#) methodologies with modern language models can significantly enhance the transparency, trust, and responsible deployment of AI systems.

TO DO

— *TO DO*

Acknowledgements

TO DO Prof. Michele Squizzato .

TO DO

TO DO

Padova, June 2026

Lorenzo Soligo

Contents

1	Introduction	1
1.1	Company	1
1.2	Stage objective	2
1.3	Project goals	2
1.4	Thesis structure	3
2	Background knowledge	4
2.1	Algorithmic analysis structure	4
2.2	Explainability	5
2.2.1	Explainability dimensions	5
2.2.2	Explanation properties and factors that facilitate understanding	5
2.2.3	SHAP analysis	6
2.3	Prompt Engineering and LLMs	7
2.3.1	Expert Personas	7
2.3.2	Familiar formats	7
2.3.3	Chain-of-Thought	9
2.3.4	Zero-Shot prompting	9
2.3.5	Multimodal prompt	10
3	ML algorithms analysis	12
3.1	Linear regression	12
3.1.1	Mathematical model	12
3.1.2	Computational complexity	12
3.1.2.1	Inference	13
3.1.2.2	Considerazioni sulla Scalabilità	13
3.1.3	Rappresentazione Interna	13
3.1.4	Vincoli sui Dati	13
3.1.4.0.1	Preprocessing per i vincoli	14
3.1.5	Capacità Predittive	14
3.1.6	Metriche per la Confidenza	15
3.1.6.0.1	Metriche Pure	15
3.1.6.0.2	Analisi dei Plot	16
3.1.7	Metriche per la Comprensione e Spiegabilità	17

3.1.7.0.1	Feature Effect	17
3.1.7.0.2	Weight Plot	17
3.1.7.0.3	Interpretazione Diretta dei Coefficienti	17
3.1.8	Limiti di Predizione	17
3.1.8.0.1	Sensibilità agli Outlier	17
3.1.8.0.2	Problemi di Extrapolazione	18
3.1.8.0.3	Instabilità Numerica in Caso di Multicollinearità	18
3.1.8.0.4	Insufficienza su Pattern Non Lineari	18
3.1.8.0.5	Collasso su Dati ad Alta Dimensionalità	18
3.1.9	Limiti di Spiegabilità	18
3.1.9.0.1	Non Linearità Nascosta nei Dati	18
3.1.9.0.2	Interazioni Non Modellate	18
3.1.9.0.3	Sensibilità al Livello di Dettaglio	18
3.1.9.0.4	Difficoltà con Feature Categoriche	18
3.1.9.0.5	Selettività	19
3.1.10	Confronto con altri Algoritmi	19
3.1.10.0.1	Vs. Regressione Logistica	19
3.1.10.0.2	Vs. Decision Tree	19
3.1.10.0.3	Vs. Lasso / Ridge	19
3.1.10.0.4	Vs. Selezione Manuale di Feature	19
3.1.10.0.5	Vs. Generalized Additive Models (GAM)	19
4	Regressione Logistica (LogR)	20
4.1	Modello	20
4.1.1	Stima dei Coefficienti	20
4.2	Complessità Computazionale	20
4.2.1	Training	20
4.2.2	Inference	21
4.2.3	Considerazioni sulla Scalabilità	21
4.3	Rappresentazione Interna	21
4.4	Vincoli sui Dati	22
4.4.1	Interpretazione Moltiplicativa dei Pesi	22
4.4.2	Assunzioni Fondamentali	22
4.5	Capacità Predittive	22
4.5.1	Punti di Forza	22
4.5.2	Punti di Debolezza	23
4.6	Metriche per la Confidenza	23
4.6.1	Metriche Pure	23

4.6.1.1	Confusion Matrix (Matrice di Confusione)	23
4.6.1.2	Accuracy	23
4.6.1.3	Sensitivity (Recall / True Positive Rate)	23
4.6.1.4	Specificity (True Negative Rate)	23
4.6.1.5	Precision	24
4.6.1.6	Recall	24
4.6.1.7	F1-Score	24
4.6.1.8	ROC Curve e AUC	24
4.6.1.9	Z-Statistic e p-value	24
4.7	Metriche per la Comprensione e Spiegabilità	25
4.7.1	Effect Plot	25
4.7.2	Weight Plot (Coefficienti)	25
4.7.3	Odds Ratio	25
4.8	Limiti di Predizione	25
4.8.1	Non Linearità	25
4.8.2	Separazione Completa	26
4.8.3	Overfitting	26
4.8.4	Sensibilità al Classe Sbilanciate	26
4.9	Limiti di Spiegabilità	26
4.9.1	Interpretazione Moltiplicativa Complessa	26
4.9.2	Dipendenza dal Contesto	26
4.9.3	Piccoli Coefficienti Difficili da Valutare	26
4.9.4	Interazioni Nascoste	27
4.10	Confronto con altri Algoritmi	27
4.10.1	Vs. Regressione Lineare su Risposta 0/1	27
4.10.2	Vs. Linear Discriminant Analysis (LDA)	27
4.10.3	Vs. Logistic Regression Multinomiale	27
4.10.4	Vs. Lasso/Ridge per Logistica	27
4.10.5	Vs. Generalized Additive Models (GAM) Logistici	27
4.10.6	Vs. Support Vector Machines (SVM)	28
4.11	Prompt	28
5	Support Vector Machine (SVM)	28
5.1	Modello	28
5.1.1	Logica dell'Algoritmo	28
5.1.2	Hard-Margin SVM (Dati Linearmente Separabili)	28
5.1.3	Soft-Margin SVM (Dati Non Separabili)	29
5.1.4	Kernel SVM (Trasformazione Non Lineare)	30

5.1.4.1	Kernel Trick	30
5.1.4.2	Kernel Comuni	30
5.1.4.3	Rappresentazione della Soluzione	31
5.2	Complessità Computazionale	31
5.2.1	Training	31
5.2.1.1	Hard-Margin SVM (Lineare)	31
5.2.1.2	Soft-Margin SVM (con variabili di slack)	31
5.2.1.3	Kernel SVM	31
5.2.1.4	Ottimizzazioni Pratiche	31
5.2.2	Inference (Previsione)	32
5.2.3	Memoria	32
5.2.4	Scalabilità: Conclusioni	32
5.3	Rappresentazione Interna	32
5.3.1	Struttura del Modello	32
5.3.2	Implicazioni per la Spiegabilità	33
5.4	Vincoli sui Dati	33
5.4.1	Linearità (Hard-Margin SVM)	33
5.4.2	Linearità (Soft-Margin SVM)	33
5.4.3	Scalabilità	33
5.4.4	Bilanciamento delle Classi	33
5.4.5	Normalizzazione delle Feature	34
5.4.6	Nessun'altra Assunzione Strutturale	34
5.5	Capacità Predittive	34
5.5.1	Punti di Forza	34
5.5.2	Punti di Debolezza	35
5.6	Metriche per la Confidenza	35
5.6.1	Metriche di Classificazione Standard	35
5.6.2	Distanza dall'Iperpiano	35
5.6.3	Platt Scaling (Probabilità Posteriori)	36
5.7	Metriche per la Comprensione e Spiegabilità	36
5.7.1	1. Identificazione dei Support Vectors	36
5.7.2	2. Pesi dei Support Vectors (α_i)	36
5.7.3	3. Feature Importance (via Permutation o SHAP)	37
5.7.4	4. Visualizzazione della Separazione (2D/3D)	37
5.7.5	5. Analisi Locale: Contrastive Explanation	37
5.8	Limiti di Predizione	38
5.8.1	Non Probabilistico per Default	38
5.8.2	Sensibilità al Tuning di Iperparametri	38

5.8.3	Sensibilità alla Normalizzazione delle Feature	38
5.8.4	Performance su Dataset Sbilanciate	38
5.8.5	Inefficienza su Dataset Grandi	38
5.9	Limiti di Spiegabilità	38
5.9.1	Opacità della Trasformazione Non Lineare (Kernel)	38
5.9.2	Interpretazione Limitata di α_i	39
5.9.3	Dipendenza da Support Vectors Arbitrari	39
5.9.4	Scarsa Tracciabilità Globale	39
5.9.5	Nessuna Spiegazione Naturale per Feature Importanza	39
5.10	Confronto con altri Algoritmi	40
5.10.1	Vs. Logistic Regression	40
5.10.2	Vs. Decision Tree	40
5.10.3	Vs. Neural Networks	41
5.10.4	Vs. Random Forest / Gradient Boosting	41
5.11	Varianti e Estensioni	41
5.11.1	1. Multi-Class SVM	41
5.11.2	2. Support Vector Regression (SVR)	42
5.11.3	3. Uno-Classe SVM (One-Class SVM)	42
5.11.4	4. Relevance Vector Machine (RVM)	42
5.12	Raccomandazioni Pratiche	42
5.12.1	Quando Usare SVM	42
5.12.2	Tuning Consigliate	43
5.13	Prompt	43
6	Decision Tree (Albero di Decisione)	43
6.1	Modello	43
6.1.1	Logica dell'Algoritmo	43
6.1.2	Criteri di Split	44
6.1.2.1	Gini Index (Classificazione)	44
6.1.2.2	Entropia (Classificazione)	44
6.1.2.3	Mean Squared Error (MSE) - Regressione	44
6.2	Complessità Computazionale	45
6.2.1	Training (Costruzione dell'Albero)	45
6.2.1.1	Nel caso generale:	45
6.2.1.2	Nella pratica:	45
6.2.1.3	Semplificazione:	45
6.2.2	Inference (Previsione)	45
6.2.3	Memoria	45

6.2.4	Considerazioni sulla Scalabilità	46
6.3	Rappresentazione Interna	46
6.3.1	Implicazioni per la Spiegabilità	46
6.4	Vincoli sui Dati	47
6.4.1	Dataset Bilanciato	47
6.4.2	Nessun Ulteriore Vincolo Strutturale	47
6.5	Capacità Predittive	47
6.5.1	Punti di Forza	47
6.5.2	Punti di Debolezza	47
6.6	Metriche per la Confidenza	48
6.6.1	Metriche di Classificazione	48
6.6.2	Metriche di Regressione	48
6.6.3	Probabilità di Foglia (Confidence Score)	48
6.7	Metriche per la Comprensione e Spiegabilità	48
6.7.1	Visualizzazione dell'Albero	48
6.7.2	Feature Importance	49
6.7.3	Spiegazione Locale (Path to Prediction)	49
6.8	Limiti di Predizione	49
6.8.1	Overfitting	49
6.8.2	Instabilità	50
6.8.3	Relazioni Lineari	50
6.8.4	Multicollinearità	50
6.9	Limiti di Spiegabilità	50
6.9.1	Alberi Complessi	50
6.9.2	Interazioni tra Feature	51
6.9.3	Bias verso Feature con Più Livelli	51
6.9.4	Difficoltà nel Comunicare Probabilità Bassa	51
6.10	Confronto con altri Algoritmi	51
6.10.1	Vs. Modelli Lineari (LR, Logistica)	51
6.10.2	Vs. Ensemble Methods (Random Forests, Gradient Boosting)	51
6.10.3	Vs. SVM (Support Vector Machine)	51
6.10.4	Vs. Neural Networks	52
6.10.5	Varianti Specializzate	52
6.10.5.1	Alberi Potati	52
6.10.5.2	Extra Trees (Extremely Randomized Trees)	52
6.10.5.3	Conditional Inference Trees	52
6.11	Prompt	52

7	Random Forest	52
7.1	Modello	52
7.1.1	Logica dell'Algoritmo	52
7.1.2	Procedura di Training	53
7.1.3	Motivazione Teorica	53
7.2	Complessità Computazionale	53
7.2.1	Training	53
7.2.2	Parallelizzazione	54
7.2.3	Inference (Previsione)	54
7.2.4	Memoria	54
7.2.5	Scalabilità: Conclusioni	54
7.3	Rappresentazione Interna	55
7.3.1	Struttura del Modello	55
7.3.2	Implicazioni per la Spiegabilità	55
7.4	Vincoli sui Dati	56
7.4.1	Nessuna Assunzione Strutturale	56
7.4.2	Bilanciamento delle Classi	56
7.4.3	Feature di Tipi Diversi	56
7.4.4	Nessun Vincolo sulla Multicollinearità	56
7.5	Capacità Predittive	56
7.5.1	Punti di Forza	56
7.5.2	Punti di Debolezza	57
7.6	Metriche per la Confidenza	57
7.6.1	Metriche di Classificazione Standard	57
7.6.2	Probabilità di Classe (Voting)	58
7.6.3	Out-of-Bag (OOB) Error	58
7.6.4	Metriche di Regressione	58
7.7	Metriche per la Comprensione e Spiegabilità	59
7.7.1	1. Feature Importance (Impurità Media)	59
7.7.2	2. Feature Importance (Permutation-Based)	59
7.7.3	3. Partial Dependence Plot (PDP)	59
7.7.4	4. Individual Conditional Expectation (ICE) Plot	59
7.7.5	5. SHAP (SHapley Additive exPlanations)	59
7.7.6	6. Single Tree Extraction	60
7.8	Limiti di Predizione	60
7.8.1	Sensibilità a Feature Dominanti	60
7.8.2	Difficoltà con Dati Molto Sbilanciati	60
7.8.3	Scarsa Extrapolazione	60

7.8.4	Performance su Feature Numeriche Pure	60
7.8.5	Sensibilità a Iperparametri	61
7.9	Limiti di Spiegabilità	61
7.9.1	Opacità Globale	61
7.9.2	Interpretazione di Feature Importance Ambigua	61
7.9.3	Interazioni Non Esplicite	61
7.9.4	Dipendenza da Iperparametri	61
7.9.5	Difficoltà di Spiegazione per Non-Esperti	62
7.10	Confronto con altri Algoritmi	62
7.10.1	Vs. Decision Tree Singolo	62
7.10.2	Vs. Logistic Regression	62
7.10.3	Vs. SVM	63
7.10.4	Vs. Gradient Boosting	63
7.10.5	Vs. Neural Networks	64
7.11	Varianti e Estensioni	64
7.11.1	1. Extra Trees (Extremely Randomized Trees)	64
7.11.2	2. Gradient Boosting (XGBoost, LightGBM, CatBoost)	64
7.11.3	3. Isolation Forest	64
7.11.4	4. Random Forest per Regressione	65
7.12	Raccomandazioni Pratiche	65
7.12.1	Quando Usare Random Forest	65
7.12.2	Hyperparameter Tuning Consigliato	65
7.12.3	Miglior Pratica di Validazione	66
7.12.4	Feature Engineering	66
7.13	Prompt	66
8	XGBoost: Extreme Gradient Boosting	66
8.1	Modello	66
8.1.1	Logica dell'Algoritmo	66
8.1.2	Obiettivo Regolarizzato	67
8.1.3	Gradient Tree Boosting (Secondo Ordine)	67
8.1.4	Scoring Function per Split	67
8.2	Complessità Computazionale	68
8.2.1	Training (Esatto)	68
8.2.2	Training (Approssimato - con Quantile Sketch)	68
8.2.3	Inference (Previsione)	68
8.2.4	Memoria	68
8.2.5	Scalabilità: Conclusioni	68

8.3	Rappresentazione Interna	69
8.3.1	Struttura del Modello	69
8.3.2	Differenze da Random Forest	69
8.3.3	Implicazioni per la Spiegabilità	69
8.4	Vincoli sui Dati	70
8.4.1	Nessuna Assunzione Strutturale	70
8.4.2	Bilanciamento delle Classi	70
8.4.3	Normalizzazione delle Feature	70
8.4.4	Gestione di Dati Sparsi	70
8.4.5	Dati Ponderati	70
8.5	Capacità Predittive	71
8.5.1	Punti di Forza	71
8.5.2	Punti di Debolezza	71
8.6	Metriche per la Confidenza	72
8.6.1	Metriche Standard di Classificazione	72
8.6.2	Margin/Distance Score	72
8.6.3	Probabilità Calibrate (Sigmoid Transform)	72
8.6.4	Early Stopping e Monitoring	72
8.7	Metriche per la Comprensione e Spiegabilità	73
8.7.1	1. Feature Importance (Gain-Based)	73
8.7.2	2. Feature Importance (Split-Based)	73
8.7.3	3. Feature Importance (SHAP)	73
8.7.4	4. Partial Dependence Plot (PDP)	73
8.7.5	5. Individual Conditional Expectation (ICE)	74
8.7.6	6. Tree Visualization	74
8.7.7	7. Explain Single Prediction (Contrastive)	74
8.8	Limiti di Predizione	74
8.8.1	Sensibilità al Tuning di Iperparametri	74
8.8.2	Difficoltà su Dati Lineari	75
8.8.3	Scarsa Generalizzazione con Molto Rumore	75
8.8.4	Extrapolazione Debole	75
8.9	Limiti di Spiegabilità	75
8.9.1	Opacità della Struttura Sequenziale	75
8.9.2	Interpretazione di Feature Importance Ambigua	75
8.9.3	Difficoltà di Comunicazione	76
8.9.4	SHAP è Computazionalmente Costoso	76
8.9.5	Instabilità di Feature Importance	76
8.10	Confronto con altri Algoritmi	76

8.10.1	Vs. Random Forest	76
8.10.2	Vs. Logistic Regression	77
8.10.3	Vs. Neural Networks	77
8.10.4	Vs. LightGBM / CatBoost	77
8.11	Varianti e Estensioni	78
8.11.1	1. XGBoost Rank (Learning to Rank)	78
8.11.2	2. XGBoost GPU	78
8.11.3	3. XGBoost per Distributed Training	78
8.11.4	4. Dart (Dropouts Meet Multiple Additive Regression Trees)	78
8.11.5	5. Linear Booster	79
8.12	Raccomandazioni Pratiche	79
8.12.1	Quando Usare XGBoost	79
8.12.2	Workflow Consigliato	79
8.12.3	Hyperparameter Tuning Dettagliato	79
8.12.3.1	Step 1: Learning Rate e Number of Rounds	79
8.12.3.2	Step 2: Tree Depth e Min Child Weight	80
8.12.3.3	Step 3: Regularizzazione (Lambda, Alpha)	80
8.12.3.4	Step 4: Column Subsampling	80
8.12.4	Feature Engineering Minimo Necessario	80
8.12.5	Debugging Overfitting	80
8.12.6	Debugging Underfitting	81
8.13	Considerazioni Etiche e Normative	81
8.13.1	GDPR Right to Explanation	81
8.13.2	Fairness e Bias	81
8.14	Prompt	81
8.15		81
9	Design and code	
	implementation	82
9.1	Tecnologie e strumenti	82
9.1.1	Tecnologia 1	82
9.1.2	Tecnologia 2	82
9.2	Progettazione	82
9.3	Design Pattern utilizzati	82
9.4	Codifica	82
10	Prompt Engineering	84

11 Conclusion	86
11.1 Consuntivo finale	86
11.2 Raggiungimento degli obiettivi	86
11.3 Conoscenze acquisite	86
11.4 Valutazione personale	86
Bibliography	88
12 Glossary	90

List of Figures

1.1 Zucchetti S.p.A. logo.	1
2.1 SHAP library logo.	7
2.2 Formats for prompting. Source: @formats-llms.	9

List of Tables

1.1 Table of the project timeline.	2
1.2 Table of project goals.	3
5.1	40
5.2	40
5.3	41
5.4	41

7.1		62
7.2		62
7.3		63
7.4		63
7.5		64
8.1		69
8.2		76
8.3		77
8.4		77
8.5		78

Chapter 1

Introduction

Introduction to the stage idea, goals, company and thesis organization. This chapter is meant to provide the necessary context for the reader to understand the motivation and the structure of the thesis.s

1.1 Company

Zucchetti S.p.A. is a leading Italian software company specializing in providing comprehensive IT solutions for businesses. Founded in 1978, Zucchetti has grown to become one of the largest software providers in Italy, offering a wide range of products and services that cater to various industries, including manufacturing, retail, healthcare, and public administration.

With a strong focus on innovation and customer satisfaction, Zucchetti is committed to helping organizations optimize their operations and achieve their business goals through cutting-edge technology. Exactly from this commit, the company has been actively involved in the development and implementation of [ML](#) and [Artificial Intelligence \(notai\)](#) solutions, aiming to enhance the capabilities of their software offerings and provide more value to their clients.



Zucchetti S.p.A. logo.

1.2 Stage objective

The internship project is centered around the exploration of [XAI](#) techniques, with a specific focus on the interpretability and comprehensibility of machine learning algorithms. The primary objective is to analyze a variety of predictive models, ranging from basic [regression](#) and [classification](#) algorithms to more complex ensemble methods, in order to evaluate their transparency and explainability. The idea is to increase the understanding of the results of the models, and to make them more accessible to non-expert stakeholders, by leveraging advanced [Prompt Engineering \(PE\)](#) principles to develop tailored prompts for [LLMs](#) that can automatically generate human-readable explanations of algorithmic behaviors and predictions.

The timeline of the project is structured as follows:

Week	Task
1st	Analysis of Linear Regression, Logistic Regression, Support Vector Machines.
2nd	Analysis of Decision Trees, Random Forests, XGBoost.
3rd	Code implementation and Prompt Engineering for the analyzed algorithms.
4th	Evaluation and documentation of the results.
5th	Analysis of RuleFit and deep dive in Random Forests.
6th	Analysis of Symbolic Regression.
7th	Code implementation and Prompt Engineering for the analyzed algorithms.
8th	Evaluation and documentation of the results, final report writing.

Table of the project timeline.

1.3 Project goals

The goals follow a notation to distinguish between necessary (N), desirable (D) and optional (O) goals. In the planning phase the goals were defined as follows:

Goal	Description
O-01	Complete and profound comprehension of the choosen algorithms, their mathematical foundations and their internal knowledge representation.
O-02	Comprension of interpretability and explainability techniques in machine learning.
O-03	Comprehension of algorithms design techniques.
O-04	Prompt generation for Large Language Models to generate human-readable explanations.
O-05	Comprehension and application of Prompt Engineering principles in the context of XAI.
D-01	Creation of a metric to evaluate the explainability of the analyzed algorithms.
D-02	Automatization of the analysis pipeline from dataset to LLM requests.
F-01	Application in real-world scenarios of the interpretability and explainability of Machine Learning models.
F-02	Explainability study of semi-supervised and unsupervised algorithms.

Table of project goals.

1.4 Thesis structure

The second chapter describes the basic concepts that fuel the project. The concepts spread from the mathematical foundations of the analyzed algorithms, the basics of explainability, to the principles of Prompt Engineering and LLMs.

The third chapter analyzes the ML algorithms, focusing on their capability, limitation and interpretability.

The fourth chapter describes the design and implementation of the analysis pipeline from dataset to LLM requests.

The fifth chapter discusses prompt engineering techniques and their application in the context of XAI.

The sixth chapter presents the conclusions of the study, the findings and the goals assessment.

Chapter 2

Background knowledge

In this chapter, we will provide the necessary background knowledge to understand the project. We will cover the mathematical foundations of the analyzed algorithms, the basics of explainability, and the principles of Prompt Engineering and LLMs.

2.1 Algorithmic analysis structure

The analysis of the algorithms is structured in a systematic way to ensure a comprehensive evaluation of their interpretability and comprehensibility as well as their functioning. The structure includes the following components:

1. **Mathematical foundations:** detailed examination of the mathematical principles underlying each algorithm, including their assumptions and theoretical properties. The focus is on the understanding of the specific [Objective Function \(objective function\)](#) that the algorithms optimize, the regularization techniques they use, and the mathematical properties that govern their behavior.
2. **Computational Complexity (computational complexity):** analysis of the computational requirements of each algorithm, including time complexities and scalability regarding both training and inference.
3. **Internal representation:** exploration of how each algorithm internally represents data and makes decisions, especially regarding [Categorical Features \(categorical features\)](#).
4. **Data assumptions:** investigation into the assumptions each algorithm makes about the data, such as linearity, independence of features, and distributional assumptions.
5. **Predictive performance and limitations:** evaluation of the predictive performance of each algorithm on relevant datasets, considering the data assumptions and mathematical foundation of the model.
6. **Metrics for prediction quality:** presentation of the most relevant metrics for evaluating the models performance, based on the task.
7. **Metrics for interpretability:** presentation of the most relevant metrics for evaluating the models interpretability, based on the task.
8. **Explainability limitations:** discussion of the limitations of explainability techniques for each algorithm, including potential pitfalls and challenges in interpreting their predictions.

2.2 Explainability

Explainability (or interpretability) of a ML model is the ability to communicate **how and why** the model made a prediction.

Contrary to intuition, explainability it's **not a binary attribute**, but a spectrum with multiple dimensions.

2.2.1 Explainability dimensions

1. Intrinsic Transparency (Model-Level Interpretability)

The model is inherently transparent and interpretable by design. The structure directly communicates how it works, such as regression coefficients or decision tree nodes. Each prediction is fully traceable, and parameters have tangible meanings. Explanations reflect human reasoning, making them coherent from a domain perspective.

It represents the best case scenario for explainability but it is often in trade-off with performance, as more complex models tend to be less transparent but more powerful. It is generally organized in tiers: High, Medium and Low transparency.

2. Global Interpretability (Model-Level Explainability)

The model is comprehensible in its entirety, allowing to understand its overall behaviour and decision-making process. Such a level of understanding it's hardly achievable and it is usually indirectly reached by comprehending the singular components of the models instead of the model as a whole.

3. Modular Interpretability

When global interpretability is not achievable, we can still understand the model by analyzing its components separately. Exploiting the model transparency, we can understand the role of each component in the decision-making process.

This concept is clear in the case of linear methods, where it is easy to understand the contribution of the single weights or in tree based models, where the splits and the structure of the tree can be analyzed separately.

4. Local Interpretability (Instance-Level Explainability)

The model might be opaque and too complex to understand globally. However, for a specific prediction, we can generate an explanation that is locally interpretable. SHAP could be used in this context to explain a particular instance obtained a certain prediction.

2.2.2 Explanation properties and factors that facilitate understanding

For human explanations to be effective, they should have certain properties that facilitate understanding and usability. The following properties can be identified from the work of Robnik-Sikonja and Bohanec 2018 and Molnar [1]:

- **Contrastive explanations**

Comparative explanations that highlight the differences between instances can be more informative than absolute explanations. The instances confrontation makes the explanation more impactful, providing a direct idea of how different factors contribute to different outcomes.

- **Cause selection**

Not all features are equally important; explanations should focus on the most influential factors rather than listing all contributing features. A small sample of the most relevant features provide a more clear explanation than a long list of all features preventing information overload.

- **Social and contextual factors**

The social environment and the audience’s expertise level should be considered when generating explanations to ensure they are appropriate and understandable.

- **Anomalies or abnormal predictions**

Highlighting anomalies or abnormal predictions can help users understand when the model is making an unusual decision, which can be crucial for trust and debugging. Features that are not largely relevant for the majority of the predictions but that have a large impact on specific predictions should be highlighted for their capability.

- **Fidelity**

The explanation should accurately reflect the model’s behavior and not introduce misleading information. A high-fidelity explanation ensures that users can trust the insights provided and make informed decisions based on them, provided that the model is accurate in its predictions.

- **General and probable**

In the absence of anomalies of specific cases, a good explanation should rely on general and probable causes that are likely to be true for most instances. This parameter can be quantified by the feature support:

$$\text{Feature Support}_i = \frac{\text{\#training instances with similar values to } x_i}{\text{\#total training instances}}$$

Generality can be misleading, as it may lead to explanations that are too broad and not specific enough to be useful. It’s important to balance generality with the impact of the explanation, ensuring that it provides meaningful insights without being overly vague. This is why usually [abnormal features](#) are highlighted, as they can provide more specific and impactful explanations.

2.2.3 SHAP analysis

SHAP (SHapley Additive exPlanations) is a method for explaining the output of machine learning models by attributing the prediction to each feature. It is based on the concept of Shapley values from cooperative game theory, which provide a way to fairly distribute the “payout” (in this case, the prediction) among the “players” (the features) based on their contribution to the prediction. SHAP values can be used to understand the importance of each feature for a specific

prediction (local interpretability) or for the model as a whole (global interpretability). SHAP provides a unified framework for interpreting various types of models and can be applied to both linear and non-linear models, making it a powerful tool for explainable AI.



SHAP library logo.

2.3 Prompt Engineering and LLMs

The following sections will present the principles used in the design of the prompts for the [LLMs](#) to generate human-readable explanations of the analyzed algorithms.

2.3.1 Expert Personas

The expert personas techniques consists in the design of prompts that describes the role that the [LLM](#) should play in the task. This technique first should guide the model in generating a more precise, coherent and relevant output.

Though the expert personas seems to be linked to a loss of accuracy in multiple benchmarks in the recent studies by Zizhao Hu [\[2\]](#), the principal use of the LLMs in this project is results evaluation and text generation. The data and metrics do not have to be generated and have only to be read directly from the prompt. The paper specifically talks about “*For tasks that depend on pretrained knowledge retrieval accuracy*” [\[2\]](#). It’s reasonable to think that in such a context, loss of accuracy is not a critical issue. In the same paper, an alignment of behavior and style to the described personas is observed. This drift in behavior is critical in the task of describing the results of an algorithm in a human-readable way, as the style of the explanation is fundamental to make it accessible to non-expert stakeholders.

2.3.2 Familiar formats

The extensive use of [LLMs](#) have led to the emergence of some familiar formats that are widely used in the design of prompts.

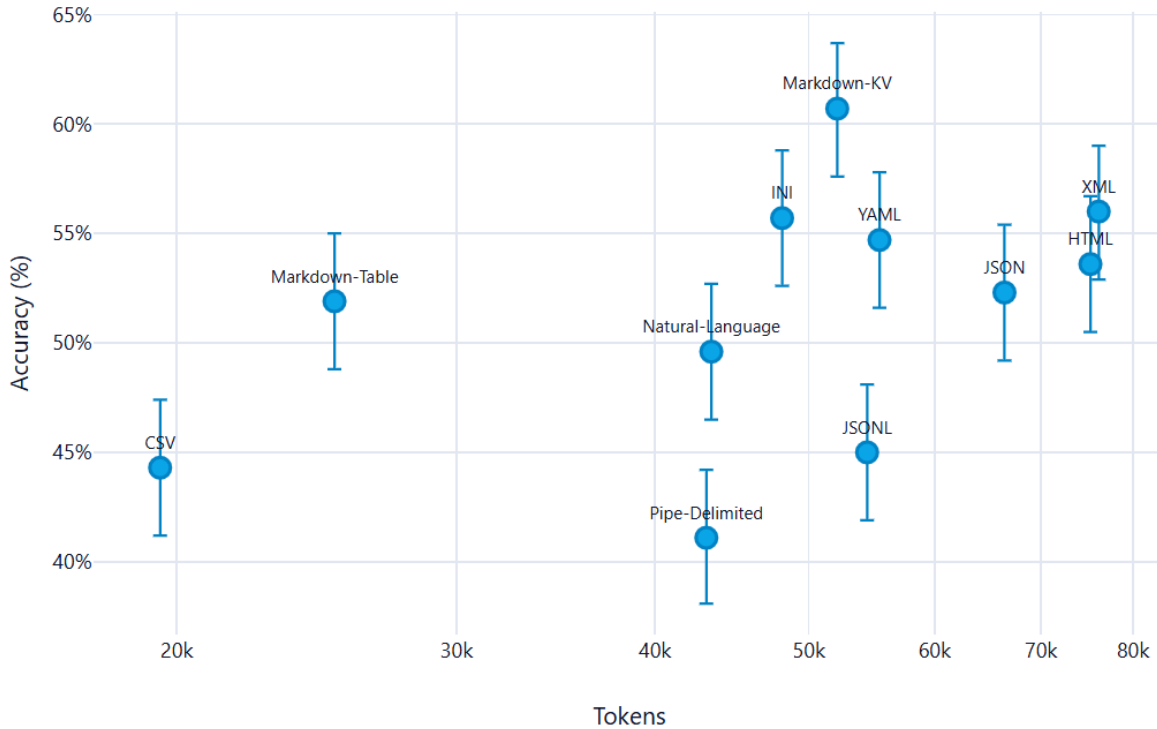
The principal features that this formats present, and that makes them particularly effective, are the following:

- Widely use in training: the more a format is used in the training of the model, the more likely is to be effective in the interpretation of content in that format.
- Clear structure: the format should have a clear and recognizable structure that allows the model to easily identify the different components of the prompt and their relationships.
- Token efficiency: the format should be designed in a way that minimizes the number of tokens used, while still conveying all the necessary information. This is particularly important in the context of LLMs, as the number of tokens used can have a significant impact on the performance and cost of the model.

From the multitude of formats in existence, some emerged for their particular accuracy and efficiency. The most valueable one is **Markdown**, which has a clear and recognizable structure that allows the model to easily identify the different components of the prompt and their relationships. The use of markdown also allows to create a clear and organized structure for the generated explanations, making them more readable and accessible to non-expert stakeholders. Another accurate format is YAML, which offers a more readable formatting than JSON and XML while still being well structured.

An alternative like CSV can be used, as it is a widely used format for tabular data and is likely to be well understood by the model. However, it's important to ensure that the CSV format is properly structured and that the necessary information is included in the prompt to allow the model to generate accurate and relevant explanations.

What follows is a summarizing image where the 11 formats taken in exam in the study[3] are presented. The y-axis is a scale of Accuracy while the x-axis a token consumption scale. It's clear that in this particular case of study, only OpenAI's GPT-4.1 nano has been tested, Markdown emerges as the most accurate format, while maintaining good token usage.



Formats for prompting. Source: @formats-llms.

2.3.3 Chain-of-Thought

Chain of Thought (CoT) prompting is a technique that consists in the design of prompts that guide the [LLMs](#) to generate a step-by-step reasoning process to reach the final answer. This technique is particularly useful in the context of this project, as it allows to generate explanations that are more coherent and relevant to the analyzed algorithms.

The additional value of the guided, step-by-step reasoning has been proven in multiple benchmarks[4], and it is particularly useful in the context of this project, as it allows to generate explanations that are more coherent and relevant to the analyzed algorithms.

In the particular context of [XAI](#) and non-technical audiences because it enhances the model's ability to provide detailed and understandable explanations. The generated answer is not just a final result, but a comprehensive explanation that breaks down the reasoning process into clear and logical steps.

2.3.4 Zero-Shot prompting

This technique consists in the design of prompts that do not include any example of the expected output. It's the simplest kind of prompting because it erases the problem of having to provide training examples. Such method is particularly necessary in the context due to the fact that an example of data and metrics for reference could be too specific and not generalized enough. An additional set of data could also confuse the model and make it less accurate or biased on

the actual result of the analysis.

The zero-shot prompting also revealed to be quite effective with some additional techniques such as the use of [Chain of Thought](#) (CoT) prompting [5].

2.3.5 Multimodal prompt

The multimodal prompting technique consists in the design of prompts that include multiple modalities, such as text, images, tables, etc. This technique is particularly useful in the context of this project, as it allows to provide the model with a richer and more comprehensive context for generating explanations.

The use of multimodal prompts can enhance the model’s ability to comprehend and analyze the results of the algorithms, as it can leverage the information provided in different formats to generate more accurate and relevant explanations. In fact, *“visual prompts effectively interact with the textual prompts, enhancing the alignment between modalities and thereby improving the model’s performance on the zero-shot instruction learning”* [6].

Chapter 3

ML algorithms analysis

In this chapter, we will analyze the machine learning algorithms used in the project, looking at the mathematical foundations, the data requirements, the predictive performance, and the interpretability of the results.

This initial analysis will guide the design and implementation of the pipeline as well as the creation of the specific prompts for the [LLMs](#) to generate human-readable explanations of the results.

3.1 Linear regression

3.1.1 Mathematical model

The linear regression model is the simplest form of regression. It assumes a linear relationship between the input features and target variable. The mathematical representation of the linear regression model is:

$$y = \sum_{i=0} \beta_i x_i \forall i \in F$$

Where F is the set of features, β are the calculated weights for each of the features, and y is the target variable.

The goal of the linear regression is to find the optimal weights β that minimize the error between the predicted values and the actual target variable. This is typically done by minimizing the [Mean Squared Error \(MSE\)](#) between the predicted values and the actual target variable. The optimization problem can be formulated as:

$$\hat{\beta} = \operatorname{argmin}_{\beta_0 \dots \beta_p} \sum_{i=1}^n (y^{(i)} - (\beta_0 + \sum_{j=1}^p \beta_j x_j))^2$$

Viene minimizzato l'errore quadratico tra la previsione e la y reale attraverso la soluzione analitica (forma chiusa):

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

dove X è la matrice dei dati aumentata della colonna di 1 per l'intercetta.

3.1.2 Computational complexity

For the resolution of the linear regression model, two appoces are commonly used: the analytical solution and the iterative optimization methods (e.g. Gradient Descent).

The analytical solution involves the construction of the matrix calculation of the inverse of the matrix

- **Complessità principale:** $O(p^2n + p^3)$
 - $O(p^2n)$ per il calcolo della matrice $X^T X$
 - $O(p^3)$ per l'inversione della matrice
- **Memoria:** $O(p^2)$ per memorizzare la matrice quadrata $X^T X$

3.1.2.1 Inference

- **Complessità:** $O(p)$ per istanza (moltiplicazione vettore-vettore)
- **Memoria:** $O(p)$ per il vettore dei coefficienti

3.1.2.2 Considerazioni sulla Scalabilità

La dipendenza $O(p^3)$ dall'inversione matrice rende il modello computazionalmente **critico quando il numero di feature è elevato** ($p \gg 1000$ diventa problematico). In questi scenari è preferibile usare metodi iterativi come Gradient Descent, che scalano meglio a costo di maggiore tempo di training.

La soluzione analitica è comunque vantaggiosa per dataset di **piccolo-medio volume** (n, p moderati) poiché:

- Garantisce convergenza in un unico passo
- Non richiede tuning di parametri (es. learning rate)
- Fornisce una soluzione globale ottimale

3.1.3 Rappresentazione Interna

La regressione lineare rappresenta internamente il modello come un **vettore di pesi** $\beta = [\beta_0, \beta_1, \dots, \beta_p]$.

Implicazioni per la spiegabilità:

- Rappresentazione **estremamente compatta e trasparente**: ogni feature ha un coefficiente direttamente interpretabile
- La previsione per una singola istanza è una combinazione lineare esplicita
- Non esiste una "scatola nera": il calcolo della predizione è completamente tracciabile
- Ogni peso corrisponde direttamente all'impatto della feature sulla predizione (impatto lineare e costante indipendentemente dal valore della feature)

Contrasto con altri modelli: Un albero di decisione rappresenta il modello come una struttura gerarchica complessa; un modello di regressione logistica introduce una trasformazione non lineare (logistica) dopo i pesi.

3.1.4 Vincoli sui Dati

La regressione lineare richiede il rispetto di **6 assunzioni fondamentali**:

1. **Linearità dei vincoli:** le relazioni tra feature e target devono essere lineari. Interazioni o relazioni non lineari devono essere introdotte manualmente (es. termini quadratici x^2 , interazioni $x_i \cdot x_j$)
2. **Normalità dei residui:** i residui $\epsilon_i = y_i - \hat{y}_i$ devono seguire una distribuzione normale. Violazioni gravi compromettono l'inferenza statistica (p-value, intervalli di confidenza)
3. **Omoschedasticità:** i residui devono avere varianza costante in tutti i livelli delle feature. In pratica, questa assunzione è **frequentemente violata**. Esempio: nel real estate, il prezzo di case molto grandi è estremamente variabile, mentre quello di case piccole è concentrato
4. **Indipendenza delle misure:** le osservazioni non devono essere correlate tra loro (es. non dati temporali consecutivi senza controllo). Dati dipendenti (come serie temporali) violano questa assunzione
5. **Feature fisse:** le feature devono essere considerate come costanti, non soggette a errori di misurazione significativi. Se le feature contengono errore di misura, i coefficienti sono distorti (attenuation bias)
6. **Assenza di multicollinearità:** le feature non devono essere fortemente correlate tra loro. Multicollinearità causa instabilità numerica durante l'inversione di $X^T X$ e pesi inflazionati in valore assoluto ma con segni incerti

3.1.4.0.1 Preprocessing per i vincoli

1. **Identificazione multicollinearità:** calcolo della matrice di correlazione tra feature o VIF (Variance Inflation Factor) per identificare feature altamente correlate. $VIF_j = \frac{1}{1-R_j^2}$

Dove R_j^2 è il coefficiente di determinazione del modello di regressione della feature j sui restanti predictor.

1. **Matrice di correlazione:** calcolo della matrice di correlazione per identificare coppie di feature altamente correlate con il coefficiente di Pearson (correlazione > 0.8 o < -0.8).

3.1.5 Capacità Predittive

Punti di forza:

- Eccellente per relazioni **linearmente separabili** tra feature e target
- Computazionalmente efficiente sia in training che in inference
- Soluzione **esatta e garantita** (non iterativa, senza convergenza incerta)

Punti di debolezza:

- **Assolutamente inadatto a pattern non lineari:** la forma funzionale rigida è un vincolo, non una feature
- Fortemente semplificativo: impone una struttura di relazione che spesso non esiste nei dati reali

- Sensibile agli **outlier**: errori grandi vengono quadrati nella loss function, amplificando l'effetto

Limitazione strutturale: Molti degli assunzioni su cui si basa risultano **non rispettati nella realtà**. L'omoschedasticità, ad esempio, è raramente verificata in problemi reali.

3.1.6 Metriche per la Confidenza

3.1.6.0.1 Metriche Pure

3.1.6.0.1.0.1 R^2 (Coefficiente di determinazione)

Quantifica quanto il modello spiega la varianza totale dei dati (range: $0 \rightarrow 1$)

$$R^2 = 1 - \frac{SSE}{SST}$$

Dove:

- $SSE = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$ è la somma dei quadrati dei residui (varianza non spiegata)
- $SST = \sum_{i=1}^n (y^{(i)} - \bar{y})^2$ è la varianza totale

Interpretazione: $R^2 = 0.85$ significa che il modello spiega l'85% della variabilità totale.

3.1.6.0.1.0.2 $\bar{R}^2$ (R^2 aggiustato)

Variazione di R^2 che penalizza l'aggiunta di feature non utili. A differenza di R^2 che cresce sempre al crescere del numero di feature, $\bar{R}^2$ diminuisce se la feature aggiunta non compensa la complessità:

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

Dove p è il numero di feature (*predittori*) e n il numero di istanze.

3.1.6.0.1.0.3 Feature Importance (t-statistic)

Misura la significatività statistica di ogni coefficiente, calcolata come il peso scalato per il suo errore standard:

$$t_{\hat{\beta}_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

Interpretazione:

- Valori assoluti grandi $\rightarrow$ la feature è statisticamente significativa nella previsione
- Cresce al crescere del peso, diminuisce all'aumentare della varianza
- Associato a un **p-value** che quantifica la probabilità che il coefficiente sia dovuto al caso

3.1.6.0.1.0.4 p-value

Per ogni coefficiente, il p-value risponde: "Se il vero coefficiente fosse 0, quale è la probabilità di osservare un valore così estremo per $t_{\hat{\beta}_j}$?". Convention: $p < 0.05$ indica significatività.

3.1.6.0.1.0.5 Mallows' Cp

Criterio per la **selezione del modello** che bilancia errore di training con complessità:

$$Cp = \frac{SSE}{\hat{\sigma}^2} - n + 2p$$

Dove $\hat{\sigma}^2$ è la stima della varianza dei residui del modello completo. Un modello ideale ha $Cp \approx p + 1$.

3.1.6.0.1.0.6 RMSE (Root Mean Squared Error)

Misura la magnitudine media degli errori, nella stessa scala della variabile target:

$$RMSE = \sqrt{\frac{SSE}{n}}$$

3.1.6.0.1.0.7 MAE (Mean Absolute Error)

Alternativa a RMSE, meno sensibile agli outlier:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

3.1.6.0.2 Analisi dei Plot

3.1.6.0.2.0.1 Actual vs Predicted

Grafico di dispersione (scatter plot) tra valori reali y_i e predetti $\hat{y}_i$.

- **Interpretazione corretta:** punti concentrati intorno alla diagonale $y = \hat{y}$
- **Anomalie:** dispersione crescente con il valore predetto (eteroschedasticità), o pattern sistematico (non linearità)

3.1.6.0.2.0.2 Istogramma dei Residui

Distribuzione dei residui $\epsilon_i = y_i - \hat{y}_i$.

- **Interpretazione corretta:** distribuzione simmetrica e campanulare (normale)
- **Anomalie:** asimmetria o code pesanti indicano violazione della normalità

3.1.6.0.2.0.3 Q-Q Plot (Quantile-Quantile)

Rappresenta i quantili empirici dei residui vs i quantili di una distribuzione normale.

- **Interpretazione corretta:** punti seguono la retta diagonale
- **Anomalie:** deviazioni sistematiche (specialmente alle code) indicano non normalità

3.1.6.0.2.0.4 Residuals vs Fitted Values

Grafico di dispersione con asse x = valori predetti $\hat{y}_i$, asse y = residui ϵ_i .

- **Interpretazione corretta:** nuvola di punti senza pattern, orizzontale attorno a 0
- **Anomalie:**
 - Pattern sistematico (curva) → non linearità
 - Varianza crescente con x → eteroschedasticità
 - Bande orizzontali → outlier o dati discreti

3.1.7 Metriche per la Comprensione e Spiegabilità

3.1.7.0.1 Feature Effect

Rappresentazione dell'impatto di ogni feature sulla previsione, calcolato come:

$$effect_j^{(i)} = \beta_j x_j^{(i)}$$

Visualizzazione: box plot per quartili (25%-75%) per ogni feature

- **Larghezza del box** = variabilità dell'effetto nei dati reali
- Una feature con box ampio ha un impatto medio elevato sulla previsione
- Utile per identificare quali feature contano di più

3.1.7.0.2 Weight Plot

Rappresentazione grafica dei coefficienti β_j ordinati per valore assoluto.

- Feature con barre lunghe hanno maggiore influenza sulla previsione
- Utile per comunicare rapidamente l'importanza relativa delle feature

3.1.7.0.3 Interpretazione Diretta dei Coefficienti

Ogni coefficiente β_j ha un'interpretazione immediata: "aumentando la feature x_j di 1 unità, la previsione cambia di β_j unità" (mantenendo le altre feature costanti).

Questa **trasparenza è il vantaggio principale** della regressione lineare per la spiegabilità rispetto a modelli più complessi.

3.1.8 Limiti di Predizione

3.1.8.0.1 Sensibilità agli Outlier

L'errore quadratico nella loss function amplifica gli outlier: un errore di 10 unità contribuisce 100 alla loss, quindi il modello tende a distorcersi per minimizzare outlier singolari.

3.1.8.0.2 Problemi di Extrapolazione

La regressione lineare assume che la relazione lineare continui al di fuori dell'intervallo osservato dei dati. Predizioni su valori di feature molto diversi dal training set sono estremamente inaffidabili.

3.1.8.0.3 Instabilità Numerica in Caso di Multicollinearità

Se le feature sono altamente correlate, la matrice $X^T X$ diventa mal-condizionata (quasi singolare), rendendo l'inversione numericamente instabile e i coefficienti instabili.

3.1.8.0.4 Insufficienza su Pattern Non Lineari

Non esiste modifica parametrica che possa far adattare un modello lineare a relazioni veramente non lineari senza inserire manualmente termini non lineari (e questo richiede conoscenza a priori della forma della relazione).

3.1.8.0.5 Collasso su Dati ad Alta Dimensionalità

Quando il numero di feature è prossimo o superiore al numero di osservazioni ($p \approx n$ o $p > n$), il modello tende a sovradattarsi, con R^2 perfetto sul training e pessimo sul test.

3.1.9 Limiti di Spiegabilità

3.1.9.0.1 Non Linearità Nascosta nei Dati

Se la vera relazione è non lineare ma viene forzata in un modello lineare, i coefficienti rappresentano una "media" della relazione, non la relazione vera. Un coefficiente positivo in media potrebbe mascherare un effetto che è positivo per alcuni valori della feature e negativo per altri.

3.1.9.0.2 Interazioni Non Modellate

La regressione lineare base assume indipendenza degli effetti delle feature: l'effetto di x_1 non dipende da x_2 . Se esistono interazioni significative, il modello le ignora (a meno che non siano aggiunte manualmente).

3.1.9.0.3 Sensibilità al Livello di Dettaglio

La scelta di quante feature includere è delicata: troppe causano overfitting e spiegabilità confusa, troppo poche rendono le spiegazioni superficiali e imprecise.

3.1.9.0.4 Difficoltà con Feature Categoricali

Le feature categoriche devono essere codificate (es. one-hot encoding), il che aumenta p e complessità interpretativa (un concetto logico diventa p variabili binarie).

3.1.9.0.5 Selettività

La capacità di identificare solo le feature veramente rilevanti può essere raggiunta in due modi:

- **Selezione manuale a priori:** affidamento a esperti di dominio
- **Modelli sparse (Lasso):** introduzione di penalità L1 che forzano molti coefficienti a 0
- **Metodi stepwise:** aggiunta/rimozione iterativa di feature

3.1.10 Confronto con altri Algoritmi

3.1.10.0.1 Vs. Regressione Logistica

- **LR:** per relazioni lineari **continue**
- **Logistica:** per classificazione binaria, produce probabilità dirette
- La logistica non è un'estensione di LR, ma un modello distinto per un problema diverso

3.1.10.0.2 Vs. Decision Tree

- **LR:** trasparente, coefficienti interpretati, ma rigida (niente non linearità)
- **DT:** flessibile, cattura non linearità e interazioni, ma rappresentazione complessa e instabile
- **Scelta:** dati lineari $\rightarrow$ LR; pattern complessi non lineari $\rightarrow$ DT

3.1.10.0.3 Vs. Lasso / Ridge

- **LR standard:** minimizza SSE senza penalità
- **Ridge:** aggiunge penalità L2 $\lambda ||\beta||_2^2$, riducendo i pesi ma mantenendo tutte le feature
- **Lasso:** aggiunge penalità L1 $\lambda ||\beta||_1$, azzerando pesi (selezione automatica di feature)
- **Quando usare:** multicollinearità o alta dimensionalità $\rightarrow$ Ridge/Lasso

3.1.10.0.4 Vs. Selezione Manuale di Feature

- **Selezione Lasso/Ridge:** automatica, basata sui dati
- **Selezione manuale:** richiede expertise di dominio, ma può catturare conoscenza esterna
- **Metodi stepwise (Forward/Backward):** compromesso automatico ma interpretabile

3.1.10.0.4.0.1 Forward Selection

Aggiunta passo-passo di feature: at ogni passo, aggiungere la feature che massimizza una metrica di controllo (es. R^2). Stop quando non c'è miglioramento significativo.

3.1.10.0.4.0.2 Backward Selection

Opposto: partire da tutte le feature, rimuovere iterativamente quella con contributo minore.

3.1.10.0.5 Vs. Generalized Additive Models (GAM)

- **LR:** effetti lineari e additivi
- **GAM:** combina LR con funzioni non lineari flessibili (splines) per ogni feature

- Il modello rimane **additivo** (effetti indipendenti) ma **non lineare** (curve invece di rette)

Chapter 4

Regressione Logistica (LogR)

4.1 Modello

La regressione logistica è un modello utilizzato per problemi di **classificazione binaria**, dove la variabile di risposta è dicotomica (es. "Sì/No", "Malato/Sano", "Default/Non-default").

Il modello combina una **combinazione lineare di feature** con una **funzione logistica (sigmoide)** per mantenere l'output tra 0 e 1 (probabilità):

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Nel caso specifico, la probabilità che l'istanza appartenga alla classe positiva ($y = 1$) è:

$$P(y^{(i)} = 1 | x^{(i)}) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^p \beta_j x_j^{(i)}))}$$

La probabilità della classe negativa è il complemento: $P(y^{(i)} = 0) = 1 - P(y^{(i)} = 1)$.

In generale, la probabilità congiunta è descritta da una distribuzione di Bernoulli:

$$P(y^{(i)}, x^{(i)}) = \{P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}\} \text{ Distribuzione Bernoulli}$$

4.1.1 Stima dei Coefficienti

A differenza della regressione lineare che minimizza l'errore quadratico, la regressione logistica **massimizza la likelihood** (probabilità osservare i dati dati i parametri):

$$L(\beta; y, X) = \prod_{i=1}^n P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}$$

In pratica si massimizza il **log-likelihood** per stabilità numerica:

$$\ell(\beta) = \sum_{i=1}^n [y^{(i)} \log(P(y^{(i)} = 1)) + (1 - y^{(i)}) \log(1 - P(y^{(i)} = 1))]$$

La massimizzazione avviene tramite **ottimizzazione iterativa** (Gradient Descent, Newton-Raphson, IRLS) poiché non esiste una soluzione analitica chiusa come nella regressione lineare.

4.2 Complessità Computazionale

4.2.1 Training

- **Complessità principale:** $O(p \cdot k \cdot n)$

- k = numero di iterazioni dell'algoritmo di ottimizzazione (solitamente 20-100)
- p = numero di feature
- n = numero di osservazioni
- Dipende fortemente dal metodo di ottimizzazione scelto
- **Memoria:** $O(p)$ per il vettore dei coefficienti + $O(n)$ per il batch

4.2.2 Inference

- **Complessità:** $O(p)$ per istanza (una moltiplicazione vettore + funzione sigmoid)
- **Memoria:** $O(p)$ per il vettore dei coefficienti

4.2.3 Considerazioni sulla Scalabilità

La **dipendenza iterativa** dalla convergenza rende la regressione logistica **meno prevedibile** della regressione lineare:

- Su dataset con feature altamente correlate, la convergenza può essere lenta (molte iterazioni)
- La separazione completa dei dati causa **divergenza numerica** (coefficienti $\rightarrow \pm\infty$)
- Per dataset **very large** ($n \gg 1M$), il training rimane trattabile grazie alla linearità della loss

Vantaggio rispetto a modelli come SVM o Neural Networks: rimane computazionalmente efficiente anche su dataset di media grandezza senza acceleratori GPU.

4.3 Rappresentazione Interna

Come la regressione lineare, il modello rappresenta internamente il modello come un **vettore di pesi** $\beta = [\beta_0, \beta_1, \dots, \beta_p]$.

Differenze rispetto a LR:

1. I coefficienti **non hanno interpretazione additiva diretta** (non è "aumenta di β_j "), ma **moltiplicativa** (vedi sezione Vincoli)
2. La **trasformazione sigmoide** non è invertibile facilmente, quindi l'effetto di una feature sulla probabilità **dipende dal valore attuale** di tutte le altre feature (non è localmente lineare come in LR)

Implicazioni per la spiegabilità:

- Rappresentazione **ancora compatta e relativamente trasparente**
- L'interpretazione è però **meno intuitiva** di LR: i pesi non corrispondono direttamente a variazioni di probabilità
- La previsione è ancora completamente tracciabile (non è una scatola nera), ma il ragionamento richiede comprensione della trasformazione logistica
- Per istanze vicino a probabilità 0.5, piccole variazioni nei pesi causano grandi variazioni nella previsione (derivata della sigmoid è massima a 0.5)

4.4 Vincoli sui Dati

4.4.1 Interpretazione Moltiplicativa dei Pesi

Dalla trasformazione della funzione logistica, gli **odds** (rapporto di probabilità) seguono una relazione **lineare sui log-odds**:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \exp\left(\beta_0 + \sum_{j=1}^p \beta_j x_j\right)$$

Aumentando di 1 unità una feature j , l'effetto moltiplicativo sugli odds è:

$$\frac{\text{odds}_{x_j+1}}{\text{odds}} = \exp(\beta_j)$$

Interpretazione pratica: Se $\beta_j = 0.5$, aumentare x_j di 1 unità **moltiplica gli odds per $e^{0.5} \approx 1.65$** , cioè aumenta del 65%. (Non aggiunge 0.5 come in LR!)

4.4.2 Assunzioni Fondamentali

1. **Linearità del log-odds:** $\log(\text{odds}) = \beta_0 + \sum_j \beta_j x_j$. La relazione è lineare nello **log-spazio degli odds**, non nello spazio della probabilità. Relazioni non lineari rimangono non catturate
2. **Linearità nei vincoli:** come LR, le feature si combinano linearmente nel log-odds. Interazioni devono essere aggiunte manualmente
3. **Separazione completa:** se una feature separa perfettamente le due classi (tutti i valori di una classe da un lato, tutti dell'altra dall'altro), il modello non può identificare un peso finito. L'algoritmo **diverge** (coefficiente $\rightarrow +\infty$ o $-\infty$)
4. **Distribuzione binomiale:** la variabile di risposta deve seguire una **distribuzione di Bernoulli** (ogni osservazione è indipendente e ha probabilità fissa)
5. **Indipendenza delle misure:** le osservazioni non devono essere correlate (no dati temporali consecutivi senza controllo)
6. **Feature fisse:** le feature devono essere considerate costanti, senza errore di misurazione
7. **Assenza di multicollinearità:** feature fortemente correlate causano instabilità dei coefficienti (come in LR, ma peggio perché l'ottimizzazione iterativa può oscillare)

Nota importante: L'omoschedasticità **non è un vincolo** come in LR, poiché i valori della risposta possono assumere solo 0 o 1 (non una distribuzione continua)

4.5 Capacità Predittive

4.5.1 Punti di Forza

- **Output probabilistico:** diversamente da un classificatore duro, fornisce una stima di confidenza della previsione
- **Efficienza computazionale:** training veloce anche su dataset moderatamente grandi

- **Interpretabilità:** i pesi, sebbene moltiplicativi, rimangono interpretabili
- **Linearità decisionale:** il confine decisionale è una retta (in 2D) o iperpiano (in p-D)

4.5.2 Punti di Debolezza

- **Non cattura non linearità:** come LR, pattern non lineari complessi rimangono invisibili al modello
- **Sensibile a separazione completa:** diverge numericamente se esiste una feature "perfetta"
- **Interazioni nascoste:** effetti di feature che dipendono l'uno dall'altro rimangono invisibili

4.6 Metriche per la Confidenza

4.6.1 Metriche Pure

4.6.1.1 Confusion Matrix (Matrice di Confusione)

Confronta le classi reali con quelle predette, generando 4 categorie:

- **TP (True Positives):** previsione positiva corretta
- **TN (True Negatives):** previsione negativa corretta
- **FP (False Positives):** errore di tipo I, previsione positiva errata
- **FN (False Negatives):** errore di tipo II, previsione negativa errata

4.6.1.2 Accuracy

Frazione di previsioni corrette:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

Limite: metrica misleading se le classi sono sbilanciate. Un classificatore che sempre predice la classe maggioritaria avrà accuracy alta

4.6.1.3 Sensitivity (Recall / True Positive Rate)

Frazione di positivi correttamente identificati:

$$Sens = \frac{TP}{TP + FN}$$

Risponde: "Su tutti i veri positivi, quanti abbiamo identificato?"

4.6.1.4 Specificity (True Negative Rate)

Frazione di negativi correttamente identificati:

$$Spec = \frac{TN}{TN + FP}$$

Risponde: "Su tutti i veri negativi, quanti abbiamo identificato?"

4.6.1.5 Precision

Frazione di previsioni positive corrette:

$$PREC = \frac{TP}{TP + FP}$$

Risponde: "Tra tutte le istanze che abbiamo predetto come positive, quante sono veramente positive?"

4.6.1.6 Recall

Alias per Sensitivity:

$$REC = \frac{TP}{TP + FN}$$

4.6.1.7 F1-Score

Media armonica tra Precision e Recall, utile quando si vuole bilanciare i due:

$$F1 = 2 \frac{PREC \cdot REC}{PREC + REC}$$

Quando usarlo: quando le classi sono sbilanciate e sia falsi positivi che falsi negativi hanno costo significativo

4.6.1.8 ROC Curve e AUC

ROC Curve: rappresentazione grafica del trade-off tra True Positive Rate (Sensitivity) sull'asse y e False Positive Rate (1 - Specificity) sull'asse x, al variare del threshold di classificazione.

- **Modello ideale:** curva passa per il punto (0,1) in alto a sinistra
- **Modello casuale:** curva segue la diagonale ($AUC = 0.5$)

AUC (Area Under the Curve): area sotto la ROC curve, quantifica la capacità globale del modello di discriminare tra le classi

- **AUC = 1.0:** discriminazione perfetta
- **AUC = 0.5:** modello casuale
- **AUC < 0.5:** peggio del caso

AUC è **invariante al threshold**, quindi è preferibile ad Accuracy per valutazioni comparative.

4.6.1.9 Z-Statistic e p-value

Analogo della t-statistic in LR, misura la significatività statistica di ogni coefficiente:

$$Z = \frac{\beta_j}{SE(\beta_j)}$$

Dove $SE(\beta_j)$ è l'errore standard del coefficiente.

- **Valori assoluti grandi di Z** $\rightarrow$ forte evidenza che il coefficiente è significativamente diverso da 0
- **p-value associato** $\rightarrow$ probabilità di osservare Z così estremo sotto l'ipotesi nulla ($\beta_j = 0$)
- **Convention:** $p < 0.05$ indica significatività

4.7 Metriche per la Comprensione e Spiegabilità

4.7.1 Effect Plot

Simile a LR, rappresenta l'effetto di una feature sulla **probabilità predetta** (non sugli odds, che è più difficile da interpretare).

Per ogni valore della feature, calcolare:

$$P(y = 1 \mid x_j = v, x_{\text{other}} = \text{media})$$

Il grafico mostra come la probabilità predetta varia col valore della feature, mantenendo le altre feature ai loro valori medi.

Vantaggio rispetto a LR: la trasformazione sigmoide rende visibile se l'effetto è principalmente presso certi valori della feature (grafico non è una retta, è una curva).

4.7.2 Weight Plot (Coefficienti)

Analogo a LR, rappresenta graficamente i coefficienti β_j ordinati per valore assoluto.

Caveat: il valore di β_j non corrisponde direttamente a una variazione di probabilità (è moltiplicativo sugli odds, non additivo sulla probabilità). Quindi il grafico ha **minore valore interpretativo** che in LR.

4.7.3 Odds Ratio

Espressione diretta dell'impatto di una feature sugli odds:

$$\text{OR}_j = \exp(\beta_j)$$

Interpretazione: "Aumentare feature j di 1 unità moltiplica gli odds per OR_j "

Esempio: se $\beta_j = 0.5$ e $\text{OR}_j = 1.65$, aumentare la feature del 65% gli odds di appartenenza alla classe positiva.

4.8 Limiti di Predizione

4.8.1 Non Linearità

Come LR, il modello non cattura pattern non lineari. La sigmoide è una trasformazione "accessoria" che non supera questo limite.

4.8.2 Separazione Completa

Se una feature separa perfettamente le classi, l'algoritmo di ottimizzazione **diverge numericamente**: il coefficiente tende a $+\infty$ o $-\infty$. Soluzioni:

- Penalizzare i pesi (Ridge o Lasso logistica)
- Rimuovere la feature (ma significa perdere informazione)
- Usare Firth's bias-reduced logistic regression (metodo specializzato)

4.8.3 Overfitting

Con un numero elevato di feature rispetto al numero di osservazioni ($p \gg n$), il modello può facilmente sovradattarsi al training set. Ridge o Lasso logistica riducono il rischio.

4.8.4 Sensibilità al Classe Sbilanciate

Il modello di default è addestrato minimizzando il likelihood globale. Se una classe è molto rara (es. 1% positivi, 99% negativi), il modello tenderà a predire sempre la classe maggioritaria. Soluzioni:

- Usare pesi di classe inversi alla frequenza (class weights)
- Ricampionare (oversampling della classe rara o undersampling della classe maggioritaria)
- Regolare il threshold di classificazione (non usare 0.5 di default)

4.9 Limiti di Spiegabilità

4.9.1 Interpretazione Moltiplicativa Complessa

A differenza di LR dove un coefficiente corrisponde a una variazione additiva, qui $\exp(\beta_j)$ è moltiplicativo e non intuitivo per la maggior parte degli utenti. Una feature che aumenta gli odds del 65% non è semplice da comunicare rispetto a "la previsione aumenta di 10 unità".

4.9.2 Dipendenza dal Contesto

L'effetto di una feature sulla probabilità predetta **dipende dai valori di tutte le altre feature**. Per una feature, l'aumento della probabilità è maggiore quando le altre feature sono tali che la probabilità predetta è intorno a 0.5 (dove la sigmoid è più ripida). Questo rende difficile fornire una spiegazione "universale" dell'effetto di una feature.

4.9.3 Piccoli Coefficienti Difficili da Valutare

Se β_j è piccolo (es. 0.01), l'effetto $\exp(0.01) \approx 1.01$ (aumento del 1%) è difficile da discernere dalla variabilità naturale. Questo crea incertezza sull'importanza reale della feature.

4.9.4 Interazioni Nascoste

Se due feature hanno un effetto congiunto non additivo, il modello base non lo cattura. Le interazioni devono essere introdotte manualmente, il che aggiunge complessità nel comunicare i risultati.

4.10 Confronto con altri Algoritmi

4.10.1 Vs. Regressione Lineare su Risposta 0/1

- **LR su dati binari:** produce previsioni fuori da $[0,1]$, ha issue di interpretazione, non è concepita per classificazione
- **Logistica:** mantiene output in $[0,1]$, fornisce probabilità direttamente interpretabili
- **Conclusione:** Logistica è sempre preferibile per problemi di classificazione binaria

4.10.2 Vs. Linear Discriminant Analysis (LDA)

Entrambi creano confini decisionali **lineari**. Tuttavia:

- **Logistica:** modella direttamente $P(y = 1 | x)$ (approccio discriminativo)
- **LDA:** modella la distribuzione congiunta di x e y (approccio generativo), assumendo normalità multivariata di x per ogni classe

Comparazione empirica: spesso producono risultati simili, ma logistica è **preferibile se:**

- Le assunzioni gaussiane sono violate (dati non normali)
- Le classi hanno covarianze molto diverse

4.10.3 Vs. Logistic Regression Multinomiale

- **Binaria:** classifica in due classi
- **Multinomiale:** estende a $K > 2$ classi
- Usa l'approccio **softmax**: per ogni classe k , stima $P(y = k)$ come funzione softmax dei pesi specifici di quella classe

4.10.4 Vs. Lasso/Ridge per Logistica

Come per LR:

- **Logistica pura:** minimizza log-likelihood senza penalità
- **Ridge logistica:** aggiunge penalità L2, riduce coefficienti, mantiene tutte le feature
- **Lasso logistica:** aggiunge penalità L1, azzeramento selettivo di coefficienti (feature selection)
- **Quando usare:** multicollinearità o separazione completa $\rightarrow$ Ridge/Lasso

4.10.5 Vs. Generalized Additive Models (GAM) Logistici

- **Logistica base:** log-odds lineari $\log(\text{odds}) = \beta_0 + \sum \beta_j x_j$

- **GAM logistica:** log-odds additivi ma non lineari $\log(\text{odds}) = \beta_0 + \sum f_j(x_j)$ dove f_j sono funzioni non lineari (es. splines)
- **Vantaggio:** preserva linearità nel log-odds (additività) ma cattura non linearità
- **Svantaggio:** la rappresentazione interna è più complessa (curve invece di rette)

4.10.6 Vs. Support Vector Machines (SVM)

- **Logistica:** output probabilistico, interpretazione diretta
- **SVM:** output hard (classe) o soft (margine), è un classificatore geometrico
- **Trade-off:** SVM ha spesso migliore capacità predittiva su dati complessi; Logistica è più interpretabile

4.11 Prompt

Chapter 5

Support Vector Machine (SVM)

5.1 Modello

5.1.1 Logica dell'Algoritmo

Un Support Vector Machine è un classificatore che cerca l'**iperpiano ottimale** che separa due classi massimizzando il **margine** (distanza tra l'iperpiano e i punti più vicini di ciascuna classe).

Per dati bidimensionali è una retta, per dati tridimensionali è un piano, per dati p-dimensionali è un **iperpiano** definito da:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p = 0$$

Dove $\beta = [\beta_1, \dots, \beta_p]$ è il **vettore normale all'iperpiano** (perpendicolare ad esso).

5.1.2 Hard-Margin SVM (Dati Linearmente Separabili)

Nel caso ideale dove i dati sono **perfettamente separabili**, l'obiettivo è trovare i coefficienti $\beta_0, \beta_1, \dots, \beta_p$ che massimizzano il **margine**.

La condizione di separazione è:

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq 1 \quad \forall i$$

Dove $y_i \in \{-1, +1\}$ è l'etichetta della classe.

La distanza tra un punto e l'iperpiano è data da:

$$r_i = \frac{y_i(\beta_0 + \sum_{j=1}^p \beta_j x_{ij})}{||\beta||}$$

Dove $||\beta|| = \sqrt{\sum_{j=1}^p \beta_j^2}$ è la **norma euclidea** del vettore dei coefficienti.

Il **margin** è la distanza minima tra l'iperpiano e il punto di training più vicino:

$$M = \frac{1}{||\beta||}$$

Per massimizzare il margine, è equivalente **minimizzare** $||\beta||$, il che si formula come un problema di ottimizzazione quadratica:

$$\min_{\beta, \beta_0} \frac{1}{2} ||\beta||^2$$

Soggetto al vincolo:

$$y_i \left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij} \right) \geq 1 \quad \forall i = 1, \dots, n$$

Motivazione: perché massimizzare il margine? L'intuizione è che un iperpiano con margine grande è più **robusto** a variazioni nei dati e generalizza meglio su dati non visti.

5.1.3 Soft-Margin SVM (Dati Non Separabili)

Nel caso realistico dove i dati **non sono linearmente separabili**, permettere che alcuni punti violino il margine:

Si introducono **variabili di slack** $\xi_i \geq 0$ che misurano quanto un punto viola il margine:

$$y_i \left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij} \right) \geq 1 - \xi_i \quad \forall i$$

Il problema di ottimizzazione diventa:

$$\min_{\beta, \beta_0, \xi} \left[\frac{1}{2} ||\beta||^2 + C \sum_{i=1}^n \xi_i \right]$$

Dove:

- **Primo termine:** penalità per complessità (piccoli coefficienti = iperpiano semplice)
- **Secondo termine:** penalità per violazioni del margine (ogni violazione costa $C \cdot \xi_i$)
- **Parametro C: iperparametro di regolarizzazione** che controlla il trade-off tra:
 - Margine grande (coefficienti piccoli)
 - Permettere violazioni (perdita di training bassa)

Interpretazione di C:

- **C grande:** il modello penalizza fortemente le violazioni → addattamento ai dati di training (rischio overfitting)
- **C piccolo:** il modello tollera violazioni → priorità al margine grande (rischio underfitting)

5.1.4 Kernel SVM (Trasformazione Non Lineare)

Il limite principale di SVM lineare è che funziona solo se i dati sono (approssimativamente) **linearmente separabili**. Per dati con pattern non lineari, SVM usa il **kernel trick**.

Idea: trasformare i dati in uno spazio di dimensionalità superiore (possibilmente infinita) dove diventano linearmente separabili. Tuttavia, computare esplicitamente la trasformazione è costoso. Il **kernel trick** consente di fare questo implicitamente.

5.1.4.1 Kernel Trick

Anziché trasformare i dati esplicitamente $\phi(x_i) = [f_1(x_i), f_2(x_i), \dots, f_m(x_i)]$ e poi calcolare prodotti scalari $\phi(x_i)^T \phi(x_j)$, usiamo una **funzione kernel** che calcola direttamente il prodotto scalare nello spazio trasformato:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Senza dover esplicitamente calcolare ϕ .

5.1.4.2 Kernel Comuni

1. Kernel Lineare

$$K(x_i, x_j) = x_i^T x_j$$

Nessuna trasformazione; ritorna alla SVM lineare.

2. Kernel Polinomiale

$$K(x_i, x_j) = (x_i^T x_j + 1)^d$$

Trasforma i dati in uno spazio di polinomi di grado d . Utile per pattern polinomiali.

3. Kernel RBF (Radial Basis Function - Gaussian)

$$K(x_i, x_j) = \exp(-\gamma ||x_i - x_j||^2)$$

Trasforma in uno spazio di dimensionalità infinita. È il kernel **più comunemente usato** perché:

- Molto flessibile, adatto a pattern complessi
- Un solo iperparametro γ (gamma) da tuning
- Buon compromesso tra complessità e performance

Interpretazione di γ :

- **γ grande:** il kernel focalizza su punti molto vicini (può causare overfitting)
- **γ piccolo:** il kernel considera punti lontani (più generale, rischio underfitting)

4. Kernel Sigmoid

$$K(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$$

Simile al kernel di una rete neurale. Meno comune.

5.1.4.3 Rappresentazione della Soluzione

La soluzione di SVM è rappresentata come una **combinazione lineare di prodotti kernel**:

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i K(x, x_i)$$

Dove:

- α_i sono i **coefficienti duali** (non direttamente i β_j)
- Solo una **frazione dei punti di training** ha $\alpha_i > 0$ — questi sono i **Support Vectors**

I support vectors sono i punti più informativi per la classificazione; i punti "facili" (ben separati) hanno $\alpha_i = 0$ e vengono ignorati.

5.2 Complessità Computazionale

5.2.1 Training

La complessità di training di SVM dipende dal metodo di ottimizzazione usato:

5.2.1.1 Hard-Margin SVM (Lineare)

$$O(n^2p) \text{ a } O(n^3p)$$

- Metodo **Quadratic Programming (QP)**: tipicamente $O(n^2p)$ o $O(n^3)$ per il solver
- Con n = numero di campioni, p = numero di feature
- La matrice del kernel è $n \times n$, quindi operazioni quadratiche su essa

5.2.1.2 Soft-Margin SVM (con variabili di slack)

$$O(n^2p) \text{ a } O(n^3p)$$

Stessa complessità; la penalità C non cambia la complessità asintotica.

5.2.1.3 Kernel SVM

$$O(n^2p) \text{ a } O(n^3)$$

Per kernel RBF, non è necessario calcolare esplicitamente ϕ (dimensioni potenzialmente infinite):

- Calcolo della matrice kernel: $O(n^2p)$ (comparare ogni coppia di punti)
- Solver QP: $O(n^3)$
- **Total**: $O(n^3)$ (dominato dal solver)

5.2.1.4 Ottimizzazioni Pratiche

Per dataset **grandi** ($n \gg 10.000$), i solver standard diventano intractabili. Soluzioni:

1. **Stochastic Gradient Descent (SGD) SVM**: $O(n p)$ ma con più iterazioni
2. **Online SVM**: aggiorna in streaming

3. **Approximate solutions:** vincolamenti per ridurre n
4. **Distributed training:** parallelizzazione su cluster

5.2.2 Inference (Previsione)

$$O(m \cdot p)$$

Dove m è il **numero di support vectors** ($m \leq n$).

La previsione richiede:

$$f(x_{\text{new}}) = \beta_0 + \sum_{i \in SV} \alpha_i y_i K(x_{\text{new}}, x_i)$$

Se il modello ha molti support vectors ($m \approx n$), inference diventa lenta. Se $m \ll n$ (case ideale), inference è veloce.

5.2.3 Memoria

$$O(n^2 + m \cdot p)$$

- Matrice kernel: $O(n^2)$
- Coefficienti: $O(m)$ (support vectors) + $O(p)$ (per ogni kernel non lineare)

Per dataset molto grandi, la memoria della matrice kernel può essere **proibitiva**.

5.2.4 Scalabilità: Conclusioni

- **Training:** cubic in n — **critico per dataset grandi**
 - Per $n > 100.000$, impraticabile senza approssimazioni
 - Per $n < 10.000$, ragionevole
- **Inference:** dipende da m (numero support vectors)
 - Se m è piccolo: veloce
 - Se $m \approx n$: lento
- **Memoria:** proibitiva per dataset enormi ($n > 1M$)

5.3 Rappresentazione Interna

5.3.1 Struttura del Modello

La rappresentazione interna di SVM è:

Support Vectors: $[x_1, x_2, \dots, x_m]$ (sottinsieme di n punti di training)
 Coefficienti: $[\alpha_1, \alpha_2, \dots, \alpha_m]$ e β_0
 Kernel: $K(\cdot, \cdot)$ (funzione)
 Previsione: $f(x) = \beta_0 + \sum_i \alpha_i y_i K(x, x_i)$

5.3.2 Implicazioni per la Spiegabilità

Contro:

- La previsione dipende da un **subset arbitrario** di punti di training (support vectors)
- Non è immediatamente chiaro **perché** un particolare punto è un support vector
- Per kernel non lineari (es. RBF), la trasformazione ϕ è implicita e non visualizzabile (dimensioni infinite)
- L'interpretazione di α_i non è intuitiva: non corrisponde direttamente a "importanza" della feature

Pro:

- Almeno i **support vectors sono identità**: puoi elencare quali punti di training hanno influenzato la previsione
- Questo è meglio di una rete neurale dove la decisione è completamente dispersa nei parametri
- Puoi analizzare i support vectors localmente per capire il ragionamento

Contrasto con altri modelli:

- **LR/LogR**: coefficienti sono interpretabili per feature (spiegabilità per feature)
- **DT**: cammino è tracciabile (spiegabilità per logica)
- **SVM**: support vectors sono tracciabili (spiegabilità per precedenti)

5.4 Vincoli sui Dati

5.4.1 Linearità (Hard-Margin SVM)

Hard-margin SVM **richiede separabilità lineare esatta**. Se i dati non sono linearmente separabili, il problema di ottimizzazione **non ha soluzione fattibile**.

Soluzione: Soft-margin SVM, che tollera violazioni.

5.4.2 Linearità (Soft-Margin SVM)

Soft-margin SVM è ancora lineare nello spazio dei dati (senza kernel). Se i dati hanno pattern non lineari, si usa kernel SVM.

5.4.3 Scalabilità

Come discusso, **training è $O(n^3)$** , quindi SVM non scala bene a dataset enormi senza approssimazioni.

5.4.4 Bilanciamento delle Classi

SVM può essere influenzato da **classi sbilanciate**:

- L'algoritmo massimizza il margine complessivo, il che potrebbe favorire la classe maggioritaria

- Se una classe è il 95% e l'altra il 5%, il margine "massimo" potrebbe consistere nel classificare tutto come la classe maggioritaria

Soluzioni:

- **Pesi di classe:** assegnare peso inverso alla frequenza (classe rara = peso alto)
- **Tuning di C:** aumentare C per la classe minoritaria
- **Ricampionamento:** oversampling della classe rara, undersampling della maggioritaria

5.4.5 Normalizzazione delle Feature

SVM usa la **distanza euclidea** ($\|x_i - x_j\|$) nel kernel, quindi è **sensibile alla scala**:

- Se una feature va da 0-1 e un'altra da 0-1.000.000, quest'ultima domina
- **Pratica standard:** normalizzare tutte le feature (es. standardizzazione z-score o min-max scaling)

5.4.6 Nessun'altra Assunzione Strutturale

A differenza di LR, SVM **non ha assunzioni** su:

- Normalità delle distribuzioni
- Omoschedasticità
- Relazioni lineari (soprattutto con kernel)

5.5 Capacità Predittive

5.5.1 Punti di Forza

1. Eccellente per Pattern Non Lineari (con Kernel)

- Kernel RBF può approssimare funzioni arbitrariamente complesse
- Spesso outperform modelli lineari su dati reali

2. Robusto a Outlier

- Soft-margin SVM usa slack variables; outlier hanno impatto limitato
- Il margine è costruito basandosi su support vectors, non su tutti i punti

3. Funziona Bene in Alte Dimensioni

- Efficace anche quando $p > n$ (numero di feature > numero di campioni)
- Modelli lineari tendono a overfitting in questi scenari

4. Principi Teorici Solidi

- Massimizzazione del margine ha fondamenti teorici (theory of VC dimension, generalization bounds)
- Garantisce che la soluzione è **globalmente ottimale** (problema convesso)

5. Versatile

- Funziona sia per classificazione che per regressione (SVR)

- Multi-classe mediante one-vs-rest o one-vs-one

5.5.2 Punti di Debolezza

1. Scarsa Interpretabilità

- Specialmente con kernel non lineari, difficile spiegare il ragionamento

2. Sensibilità a Iperparametri

- Scelta di C (soft-margin) e γ (RBF kernel) critica
- Tuning male = performance disastrose

3. Inefficienza su Dataset Grandi

- Complessità training $O(n^3)$ — impraticabile per $n \gg 100.000$
- Anche inference può essere lenta se molti support vectors

4. Non Probabilistico

- Output è una classificazione hard (classe -1 o $+1$)
- Per ottenere probabilità, è necessario post-processing (Platt scaling)

5. Multicollinearità Non Gestita

- Se feature sono fortemente correlate, performance può deteriorarsi
- Richiede feature selection o regolarizzazione

5.6 Metriche per la Confidenza

5.6.1 Metriche di Classificazione Standard

Stesse metriche di LogR e DT:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Precision:** $TP / (TP + FP)$
- **Recall/Sensitivity:** $TP / (TP + FN)$
- **Specificity:** $TN / (TN + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

5.6.2 Distanza dall'Iperpiano

Misura della **confidenza specifica di SVM**:

$$d_i = y_i \left(\beta_0 + \sum_{j=1}^p \beta_j x_{ij} \right)$$

Questa è la **distanza (segnata) del punto x_i dall'iperpiano**, normalizzata da $||\beta||$:

$$\text{Distanza Normalizzata} = \frac{d_i}{||\beta||}$$

Interpretazione:

- $d_i > 1$: punto ben classificato, lontano dall'iperpiano (confidenza alta)
- $d_i \approx 1$: punto sul margine (confine della decisione)
- $0 < d_i < 1$: punto nella regione di slack (violazione del margine)
- $d_i < 0$: punto classificato erroneamente

Questa distanza è un **proxy per la confidenza della previsione**: punti lontani dall'iperpiano sono predetti con confidenza maggiore.

5.6.3 Platt Scaling (Probabilità Posteriori)

SVM non produce probabilità di default. Per ottenere stime di probabilità $P(y = 1 | x)$, si usa **Platt scaling**:

$$P(y = 1) = \frac{1}{1 + \exp(A \cdot d + B)}$$

Dove A, B sono parametri appresi su un validation set, calibrando la distanza dell'iperpiano a probabilità.

Limite: aggiunge complessità e calibrazione; le probabilità risultanti non hanno la stessa garanzia teorica della regressione logistica.

5.7 Metriche per la Comprensione e Spiegabilità

5.7.1 1. Identificazione dei Support Vectors

Il set dei support vectors è la **feature più interpretabile** di SVM:

Support Vectors: [istanza_3, istanza_15, istanza_42, ...]

Rappresentano i punti di training che hanno **influenzato veramente la decisione**. Punti ben separati hanno $\alpha_i = 0$ e sono ignorati.

Utilizzo pratico:

- Analizzare i support vectors per capire quali punti sono "difficili" (contorno decisionale)
- Visualizzare support vectors vs punti di background per ispezionare il modello
- Se molti support vectors = modello complesso; se pochi = modello semplice

5.7.2 2. Pesi dei Support Vectors (α_i)

I coefficienti $\alpha_i > 0$ associati ai support vectors indicano il loro "peso" nella decisione:

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i y_i K(x, x_i)$$

Interpretazione:

- α_i grande = il support vector x_i ha grande influenza

- α_i piccolo = il support vector x_i ha piccola influenza

Caveat: interpretazione limitata per kernel non lineari, perché non è immediatamente chiaro cosa stia facendo il kernel.

5.7.3 3. Feature Importance (via Permutation o SHAP)

SVM non produce naturalmente feature importance. Soluzioni:

Permutation Importance:

- Permutare casualmente una feature nei dati di test
- Misurare il degrado in performance
- Grandi degni = feature importante

SHAP (SHapley Additive exPlanations):

- Assegnare ogni output tra le feature usando valori Shapley
- Applicabile a qualsiasi modello (SVM, NN, etc.)
- Computazionalmente costoso pero produce spiegazioni teoricamente fondate

5.7.4 4. Visualizzazione della Separazione (2D/3D)

Per dataset a bassa dimensionalità, visualizzare:

- Punti di training e loro classi
- Iperpiano di decisione (retta in 2D, piano in 3D)
- Margine (linee parallele a distanza $M = 1 / ||\beta||$)
- Support vectors (segnati diversamente)

Limite: per $p > 3$, non è possibile visualizzare direttamente. Usare proiezione PCA per ridurre dimensioni e visualizzare.

5.7.5 5. Analisi Locale: Contrastive Explanation

Per una previsione di istanza x_{new} :

- Trovare i support vectors più vicini (usando la metrica del kernel)
- Mostrare come differiscono da x_{new}

Esempio:

Istanza X è classificata come Positiva perché simile al Support Vector SV_5 (che è Positivo), diversa dal Support Vector SV_12 (che è Negativo).

Differenze principali:

- X.Income > SV_5.Income (simile)
- X.Age < SV_12.Age (dissimile)

5.8 Limiti di Predizione

5.8.1 Non Probabilistico per Default

SVM output una classificazione hard (+1 o -1), non una probabilità:

- Non ottenere stime naturali di incertezza
- Platt scaling produce probabilità, ma richiede calibrazione aggiuntiva

Conseguenza: per problemi dove la probabilità importa (es. medicina), SVM deve essere post-processato.

5.8.2 Sensibilità al Tuning di Iperparametri

Performance di SVM è **molto sensibile** a:

- **C (soft-margin):** piccolo C = underfitting; grande C = overfitting
- **γ (RBF kernel):** piccolo γ = generale; grande γ = specifico (overfitting)

Conseguenza: il tuning di iperparametri (grid search, random search, Bayesian optimization) è **mandatorio**, il che aggiunge complessità.

5.8.3 Sensibilità alla Normalizzazione delle Feature

Senza normalizzazione, feature con scale grandi dominano. SVM è **molto sensibile** a questo.

Mitigazione: normalizzare sempre le feature (standardizzazione z-score è standard).

5.8.4 Performance su Dataset Sbilanciate

Come discusso, SVM tende a favorire la classe maggioritaria.

Mitigazione: usare class weights (inversamente proporzionali alla frequenza).

5.8.5 Inefficienza su Dataset Grandi

Complessità $O(n^3)$ rende SVM **impraticabile per $n \gg 100.000$** .

Mitigazioni possibili:

- Usare approssimazioni (Nyström approximation, stochastic gradient descent)
- Sottocampionare per training, validare su full dataset
- Usare alternative scalabili (logistic regression, gradient boosting)

5.9 Limiti di Spiegabilità

5.9.1 Opacità della Trasformazione Non Lineare (Kernel)

Con kernel RBF, i dati sono trasformati in uno **spazio di dimensionalità infinita**:

$$K(x, x') = \exp(-\gamma ||x - x'||^2)$$

Questa trasformazione $\phi(x)$ è **implicita e non visualizzabile**. Conseguenze:

- Non puoi interpretare cosa stia facendo il modello nello spazio trasformato
- L'effetto delle feature sulla previsione è **nascosto dietro il kernel**
- Spiegare una previsione richiede capire il kernel, che è non intuitivo per i non-esperti

5.9.2 Interpretazione Limitata di α_i

I pesi α_i dei support vectors non hanno interpretazione diretta:

- Non significano "questa feature è importante"
- Non significano "questo support vector cambia la probabilità del 10%"
- Sono artefatti dell'ottimizzazione quadratica, non quantità interpretabili

5.9.3 Dipendenza da Support Vectors Arbitrari

Quali punti diventano support vectors dipende dalla **geometria dello spazio e dall'iperparametro C**:

- Cambiar C $\rightarrow$ diversi support vectors
- Cambiar dataset leggermente $\rightarrow$ diversi support vectors
- Non è evidente **perché** un punto è un support vector (è dovuto alla posizione geometrica, ma questa non è facilmente comunicabile)

5.9.4 Scarsa Tracciabilità Globale

Mentre puoi identificare i support vectors, **tracciare il ragionamento globale è difficile**:

- Non puoi dire "la feature X è il driver principale della classificazione" (come in LR)
- Non puoi tracciare una sequenza logica (come in DT)
- Rimane una "scatola nera" anche dopo aver visto i support vectors

5.9.5 Nessuna Spiegazione Naturale per Feature Importanza

A differenza di:

- **LR**: coefficienti diretti
- **DT**: feature importance dai split

SVM non produce feature importance naturale. Devi usare tecniche esterne (SHAP, permutation importance).

5.10 Confronto con altri Algoritmi

5.10.1 Vs. Logistic Regression

Aspetto	LogR	SVM
Output	Probabilità	Classe (distanza)
Interpretabilità	★★★★★ Alta	★★★ Bassa
Pattern Non-Lineare	★ (no kernel)	★★★★★ (con kernel RBF)
Scalabilità	★★★★★ $O(n \cdot p \cdot k)$	★★★ $O(n^3)$
Teoria	Probabilistica	Geometrica (margine)
Quando usare	Interpretabilità critica	Pattern non-lineare complesso

5.10.2 Vs. Decision Tree

Aspetto	DT	SVM
Interpretabilità	★★★★★ Altissima	★★★ Bassa
Pattern Non-Lineare	★★★★★ Naturale	★★★★★ Con kernel
Scalabilità	★★★★★ $O(n \cdot p \cdot \log n)$	★★★ $O(n^3)$
Robustezza Outlier	★★★ (sensibile)	★★★★★ (robusto)
Stabilità	★★★ (instabile)	★★★★★ (stabile)
Quando usare	Massima interpretabilità	Pattern complessi + stabilità

5.10.3 Vs. Neural Networks

Aspetto	NN	SVM
Capacità	★★★★★ Illimitata	★★★★★ Molto buona
Interpretabilità	★ Scatola nera	★★ Leggermente migliore
Scalabilità	★★★★★ Eccellente	★★ Scarsa
Data Requirement	★ (molto dati)	★★★★★ (meno dati)
Training	Lungo, GPU-friendly	Veloce ma $O(n^3)$
Teoria	Inesistente	Solida
Quando usare	Big data + capacità illimitata	Data medio + interpretabilità

5.10.4 Vs. Random Forest / Gradient Boosting

Aspetto	Ensemble	SVM
Performance	★★★★★ Spesso migliore	★★★★★ Buona
Interpretabilità	★★ (migliore di NN)	★★ Simile
Scalabilità	★★★★ (meglio di SVM)	★★
Robustezza Iperparametri	★★★★ Meno sensibile	★★ Molto sensibile
Implementazione	Easy (sklearn)	Easy (sklearn)
Quando usare	Predizione è priorità #1	Stabilità, spiegabilità, dati piccoli

5.11 Varianti e Estensioni

5.11.1 1. Multi-Class SVM

SVM è originariamente binario. Per multi-classe:

One-vs-Rest (OvR):

- Creare K classificatori binari ($K = \text{numero di classi}$)
- Ogni classificatore separa una classe dal resto
- Classe finale = massima confidenza

One-vs-One (OvO):

- Creare $K(K-1)/2$ classificatori binari (uno per coppia di classi)
- Voting: conta quante volte ogni classe è predetta
- Più computazionalmente costoso, spesso più accurato

5.11.2 2. Support Vector Regression (SVR)

Estensione di SVM per **regressione** (output continuo):

- Anziché maximizzare il margine di classificazione, minimizzare l'errore di predizione
- Introduce un'**epsilon-insensitive loss**: errori piccoli $< \epsilon$ sono tollerati
- Utile per predizione continua con gestione robusta di outlier

5.11.3 3. Uno-Classe SVM (One-Class SVM)

Per problemi di **anomaly detection** / **outlier detection**:

- Imparare un "confine" intorno ai dati normali
- Punti fuori da questo confine sono anomalie
- Usa soft-margin con una frazione ν di campioni come support vectors

5.11.4 4. Relevance Vector Machine (RVM)

Variante Bayesiana di SVM:

- Produce probabilità di output come LogR
- Spesso meno support vectors (più sparso)
- Trade-off: meno efficienza di SVM, più interpretabilità

5.12 Raccomandazioni Pratiche

5.12.1 Quando Usare SVM

Ottime scelte:

- Dataset di piccolo-medio volume ($n < 100.000$)
- Pattern non lineari complessi
- Necessità di stabilità e robustezza
- Dati ad alta dimensionalità ($p > n$)
- Outlier presenti nel dataset

Cattive scelte:

- Dataset molto grandi ($n > 1.000.000$)

- Interpretabilità è critica per decisioni (es. medicina, finanza regolata)
- Tuning di iperparametri non è possibile
- Output probabilistico naturale necessario

5.12.2 Tuning Consigliate

1. **Normalizzare sempre le feature** (z-score o min-max)
2. **Griglia di ricerca per C e γ :**
 - C: [0.1, 1, 10, 100, 1000]
 - γ : [0.001, 0.01, 0.1, 1, 'auto']
3. **Cross-validation:** 5-10 fold per valutare
4. **Class weights:** se classi sbilanciate, usare pesi inversi
5. **Kernel:** iniziare con RBF; provare polinomiale se pattern sospetto

5.13 Prompt

Chapter 6

Decision Tree (Albero di Decisione)

6.1 Modello

Un albero di decisione è un modello che divide ricorsivamente lo spazio dei dati in regioni rettangolari, creando una **struttura gerarchica di nodi decisionali**.

6.1.1 Logica dell'Algoritmo

L'algoritmo costruisce l'albero utilizzando un approccio **greedy dall'alto verso il basso** (top-down):

1. Inizio con tutte le osservazioni nel nodo radice
2. Ad ogni nodo, scegliere la feature e il threshold che **massimizzano la purezza** (classificazione) o **minimizzano la varianza** (regressione)
3. Dividere ricorsivamente i nodi figli fino a criteri di stop (profondità massima, numero minimo di campioni, purezza perfetta)

Il risultato è una **struttura ad albero** dove:

- **Nodi interni:** contengono regole decisionali (feature $\leq$ threshold?)
- **Foglie:** contengono predizioni (classe o valore numerico)

6.1.2 Criteri di Split

6.1.2.1 Gini Index (Classificazione)

Misura la **purezza di un nodo** per la classificazione:

$$\text{Gini } (D) = 1 - \sum_{i=1}^K p_i^2$$

Dove p_i è la proporzione di istanze appartenenti alla classe i nel nodo.

- **Gini = 0:** nodo puro (tutte le istanze della stessa classe)
- **Gini = 1 - 1/K:** nodo completamente impuro (distribuito uniformemente)
- **Per K=2 (binario):** massimo = 0.5

Information Gain: al fare uno split, il guadagno è misurato come la riduzione di Gini media pesata:

$$\text{IG} = \text{Gini (padre)} - \sum_{\text{figlio}} \frac{|D_{\text{figlio}}|}{|D|} \text{Gini (figlio)}$$

6.1.2.2 Entropia (Classificazione)

Misura l'**incertezza informativa** di un nodo:

$$\text{Entropia } (D) = - \sum_{i=1}^K p_i \log_2(p_i)$$

- **Entropia = 0:** nodo puro
- **Entropia = $\log_2(K)$:** nodo completamente impuro
- **Per K=2:** massimo = 1

Information Gain (con Entropia): il guadagno è la riduzione di entropia:

$$\text{IG} = \text{Entropia (padre)} - \sum_{\text{figlio}} \frac{|D_{\text{figlio}}|}{|D|} \text{Entropia (figlio)}$$

Nota: Gini e Entropia producono risultati simili in pratica; Gini è computazionalmente più efficiente. L'Information Gain è usato nelle prime implementazioni di Decision Tree (CART).

6.1.2.3 Mean Squared Error (MSE) - Regressione

Misura la **varianza all'interno di un nodo** per problemi di regressione:

$$\text{MSE } (D) = \frac{1}{|D|} \sum_{i=1}^{|D|} (y_i - \bar{y})^2$$

Dove $\bar{y} = \frac{1}{|D|} \sum_{i=1}^{|D|} y_i$ è la media dei target nel nodo.

Lo split è scelto per **minimizzare la somma pesata di MSE** nei nodi figli.

6.2 Complessità Computazionale

6.2.1 Training (Costruzione dell'Albero)

La complessità dipende dalla **profondità massima** dell'albero e dal numero di feature:

6.2.1.1 Nel caso generale:

$$O(n \cdot p \cdot \log(n) \cdot d)$$

Dove:

- n = numero di osservazioni
- p = numero di feature
- $\log(n)$ $\approx$ profondità dell'albero (se bilanciato)
- d = profondità effettiva ($\leq \log(n)$ per albero bilanciato)

6.2.1.2 Nella pratica:

- Per ogni nodo, scansionare tutte le p feature
- Per ogni feature, ordinare i dati (o usare binning per velocizzare) $\rightarrow O(n \log(n))$
- Questo accade ad ogni profondità dell'albero
- Se l'albero è bilanciato, profondità $\approx \log(n)$

6.2.1.3 Semplificazione:

Un albero completamente bilanciato ha complessità $O(n \cdot p \cdot \log^2(n))$

Nota importante: Un albero **completamente sviluppato** (no pruning) su n campioni può avere profondità fino a n (albero degenerato a lista), con complessità $O(n^2 \cdot p)$ — questo è un limite teorico, ma dimostra il rischio di overfitting.

6.2.2 Inference (Previsione)

$$O(d)$$

Dove d è la **profondità dell'albero**. Basta attraversare il cammino dalla radice alla foglia.

- Per alberi bilanciati: $O(\log(n))$
- Per alberi degenerati: $O(n)$

6.2.3 Memoria

$$O(n \cdot d + p)$$

- Memorizzazione dell'albero stesso: proporzionale al numero di nodi
- In un albero bilanciato: $O(n)$ nodi nel peggiore, $O(2^d) = O(n)$ nel medio
- Spazio per i dati di training: $O(n \cdot p)$

6.2.4 Considerazioni sulla Scalabilità

- **Vantaggi:**

- Training veloce per **piccoli-medi dataset**
- Inference molto veloce anche su dataset grandi
- Parallelizzabile a livello di feature (considerare split in parallelo per ogni feature)

- **Svantaggi:**

- Man mano che n cresce, il tempo quadratico $O(n^2)$ nel caso degenerato diventa problematico
- Per dataset enormi, tecniche di **gradient-based tree** (XGBoost, LightGBM) sono preferibili, che usano binning per ridurre $O(n \log n)$ a $O(n)$ per feature

6.3 Rappresentazione Interna

Un decision tree è rappresentato internamente come una **struttura ad albero ricorsiva**:

```
Nodo(feature=x1, threshold=5.5, left=Nodo(...), right=Nodo(...))
```

Ogni nodo contiene:

- **Condizione:** quale feature e quale threshold
- **Puntatori ai figli:** rami sinistro e destro
- **Informazioni di predizione:** classe modale (classificazione) o valore medio (regressione)

6.3.1 Implicazioni per la Spiegabilità

Vantaggi:

- La struttura è **completamente esplicita e navigabile**
- Ogni previsione ha un **tracciamento completo**: seguire il cammino dalla radice alla foglia spiega perfettamente come è stata raggiunta
- Non esiste ambiguità o "magia": è una sequenza di confronti

Svantaggi:

- **Alberi profondi o complessi** diventano difficili da interpretare (decine o centinaia di nodi)
- Le interazioni tra feature sono **implicite nella struttura**: se feature A e B interagiscono, l'albero le cattura tramite split in serie, ma questo non è ovvio dal grafico
- **Instabilità**: piccoli cambiamenti nei dati possono portare a strutture completamente diverse (vedi sezione Limiti)

Rispetto a modelli lineari (LR, logistica), la rappresentazione è **meno compatta ma più complessa**.

6.4 Vincoli sui Dati

6.4.1 Dataset Bilanciato

I decision tree possono essere **influenzati da classi sbilanciate**:

- L'algoritmo greedy tende a favorire split che massimizzano il puro della classe **maggioritaria**
- Su un dataset con 95% negativi e 5% positivi, un albero potrebbe diventare molto profondo per catturare i pochi positivi
- **Conseguenza**: scarsa capacità di classificazione sulla classe minoritaria (FN altissimi)

Soluzioni:

- Assegnare **pesi di classe** inversi alla frequenza
- Ricampionare (oversampling della classe rara, undersampling della maggioritaria)
- Usare metriche di valutazione appropriate (F1, non Accuracy)

6.4.2 Nessun Ulteriore Vincolo Strutturale

A differenza della regressione lineare e logistica, gli alberi di decisione **non hanno assunzioni** su:

- Linearità
- Normalità di distribuzioni
- Omoschedasticità
- Indipendenza di feature

Questo è un vantaggio (flessibilità), ma anche un rischio (overfitting senza controllo).

6.5 Capacità Predittive

6.5.1 Punti di Forza

- **Pattern non lineari**: cattura relazioni complesse e non lineari che modelli lineari non vedono
- **Interazioni automatiche**: le interazioni tra feature sono catturate naturalmente dalla struttura
- **Feature categoriche**: gestisce feature categoriche senza necessità di encoding
- **Nessuna assunzione**: nessun prerequisito su distribuzione o linearità dei dati

6.5.2 Punti di Debolezza

- **Pattern lineari**: su dati puramente lineari, gli alberi sono **inefficienti e imprecisi**
 - Avanzano tramite split ortogonali, creando funzioni a gradini
 - La predizione cambia bruscamente al passaggio di un threshold
 - Un modello lineare semplice avrebbe R^2 più alto con meno parametri

- **Dati sparsi ad alta dimensionalità:** con molte feature e pochi campioni, il rischio di overfitting è massimo

6.6 Metriche per la Confidenza

6.6.1 Metriche di Classificazione

Utilizza le stesse metriche della **Regressione Logistica**:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$
- **Sensitivity / Recall:** $TP / (TP + FN)$
- **Specificity:** $TN / (TN + FP)$
- **Precision:** $TP / (TP + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

6.6.2 Metriche di Regressione

Per alberi di regressione:

- **MSE:** errore quadratico medio
- **RMSE:** radice quadrata di MSE
- **MAE:** errore assoluto medio
- **R²:** frazione di varianza spiegata

6.6.3 Probabilità di Foglia (Confidence Score)

Un vantaggio specifico degli alberi è che forniscono naturalmente una **stima di confidenza della previsione**:

$$\text{Confidence} = \frac{\# \text{ istanze della classe predetta nella foglia}}{\# \text{ istanze totali nella foglia}}$$

Se una foglia contiene 100 campioni e 95 appartengono alla classe positiva, la confidenza della previsione "positivo" è 0.95.

Utilizzo: si può filtrare le predizioni per accuratezza: accettare solo previsioni con confidenza > 0.8 .

6.7 Metriche per la Comprensione e Spiegabilità

6.7.1 Visualizzazione dell'Albero

Il principale strumento è la **visualizzazione grafica dell'albero**:

- Ogni nodo mostra la condizione di split e la distribuzione delle classi
- Foglie mostrano la previsione e il numero di campioni

Limite: su alberi profondi (>5-6 livelli), la visualizzazione diventa illeggibile

6.7.2 Feature Importance

Misura quante volte una feature viene utilizzata come criterio di split e quanto riduce l'impurità media:

$$\text{Importance}_j = \frac{\sum_{\text{nodi con feature } j} (\text{IG}_{\text{nodo}}) \times (\text{n campioni nodo})/n}{\text{somma su tutti i nodi}}$$

Interpretazione: feature con importanza alta sono state cruciali per le decisioni dell'albero.

6.7.3 Spiegazione Locale (Path to Prediction)

Per una singola istanza:

1. Tracciare il cammino dalla radice alla foglia
2. Elencare tutti i confronti (feature $\leq$ threshold) che hanno determinato la previsione

Esempio:

Istanza X predetta come "Sì" perché:
- Age $\leq$ 35 ✓
- Income $>$ 50000 ✓
- CreditScore $\leq$ 700 ✓
→ Foglia: Sì (95 istanze, 85 positive)

Questo è un **vantaggio enorme per la spiegabilità** rispetto a modelli "scatola nera". Ogni previsione è completamente tracciabile.

6.8 Limiti di Predizione

6.8.1 Overfitting

Il limite **principale** degli alberi di decisione. Senza controllo:

- L'albero continua a dividersi fino a quando ogni foglia è pura (una sola classe)
- Su dataset piccoli, ogni campione potrebbe trovarsi in una foglia propria
- **Risultato:** $R^2 = 1.0$ sul training set, ma pessimo su test set

Soluzioni:

- **Pruning:** rimuovere nodi che non migliorano significativamente la generalizzazione
- **Limiti sulla crescita:** profondità massima, numero minimo di campioni per split, impurità minima per split
- **Ensemble methods:** Random Forests e Gradient Boosting riducono l'overfitting combinando più alberi

6.8.2 Instabilità

Piccoli cambiamenti nei dati di training possono causare **grandi cambiamenti nella struttura dell'albero**:

- Cambiare 1-2 campioni può portare a split completamente diversi
- L'ordine dei campioni non importa, ma la composizione sì (alta varianza)

Conseguenza: il modello è difficile da fidarsi se nuovi dati sono appena diversi dal training

Soluzione: ensemble methods mitigano questo problema

6.8.3 Relazioni Lineari

Su pattern puramente lineari, gli alberi sono **inefficienti**:

Esempio: $y = 2x_1 + 3x_2 + 1$

Un albero dovrà creare molti split ortogonali per approssimare la retta:

```
If x1 <= 5: ...  
  If x2 <= 3: ...  
    If x1 <= 4.5: ...  
      ...
```

Mentre un modello lineare cattura la relazione con due coefficienti.

Risultato: errore più alto, overfitting per compensare

6.8.4 Multicollinearità

I decision tree sono **meno sensibili** della regressione lineare a feature correlate, ma non immuni:

- Tendono a **scegliere una feature** del gruppo correlato (la "prima" a ridurre impurità)
- Ignor le altre, perdendo potenzialmente informazione complementare
- Se il dataset cambia leggermente, un'altra feature potrebbe essere scelta, causando instabilità

Conseguenza: modelli potenzialmente diversi su dataset simili

6.9 Limiti di Spiegabilità

6.9.1 Alberi Complessi

Un albero con molti nodi (es. 100+ nodi) diventa **difficile da interpretare visualmente**:

- Non riesci più a tenere in mente l'intero modello
- Le interazioni tra feature, sebbene catturate, non sono evidenti
- È ancora tracciabile per una singola istanza, ma difficile capire il "ragionamento globale" del modello

6.9.2 Interazioni tra Feature

L'albero cattura le interazioni (feature A influenza l'effetto di feature B), ma **non le rende esplicite**:

- Una foglia raggiunta dopo split $[A \leq 5] \rightarrow [B > 10]$ implica un'interazione
- Ma non è ovvio dal grafico che A e B interagiscono
- Per modelli lineari, le interazioni sono esplicite se aggiunte manualmente

6.9.3 Bias verso Feature con Più Livelli

I decision tree **tendono a preferire feature categoriche con molti valori unici**, perché hanno più opportunità di split e quindi di ridurre impurità:

Esempio: su dataset con una feature "Città" (100 città), l'albero potrebbe scegliere molteplici split su Città, mentre feature numeriche continueranno a usare threshold.

Conseguenza: un modello che sembra dipendere dalla Città, quando in realtà potrebbe essere meno importante di un'altra feature meno "versatile".

Mitigation: limitare la profondità e usare feature importance ponderata.

6.9.4 Difficoltà nel Comunicare Probabilità Bassa

Se una foglia predice "Sì" ma il 60% dei campioni sono "No", la confidenza è 0.4.

Comunicare "il modello dice Sì, ma con confidenza 0.6" è meno intuitivo che dire "la probabilità stimata è 60%" (come fa la logistica).

6.10 Confronto con altri Algoritmi

6.10.1 Vs. Modelli Lineari (LR, Logistica)

- **Alberi:** catturano non linearità e interazioni, ma complessi e instabili
- **Lineari:** trasparenti e stabili, ma rigidi
- **Scelta:** dati con pattern complessi $\rightarrow$ alberi; dati lineari o quando interpretabilità è critica $\rightarrow$ modelli lineari

6.10.2 Vs. Ensemble Methods (Random Forests, Gradient Boosting)

- **Singolo albero:** interpretabile, veloce, ma prone a overfitting
- **Ensemble:** combina più alberi, miglior predizione e stabilità, ma meno interpretabile
- **Trade-off:** interpretabilità vs accuratezza
- **Quando usare:** per problemi critici dove performance conta più di interpretabilità $\rightarrow$ ensemble

6.10.3 Vs. SVM (Support Vector Machine)

- **Alberi:** output naturale di classe/probabilità, tracciamento facile

- **SVM:** output geometrico (margine), "scatola nera" per l'interpretazione
- **Trade-off:** SVM spesso migliore accuratezza su dati complessi, alberi più interpretabili

6.10.4 Vs. Neural Networks

- **Alberi:** interpretabili, nessun tuning di hyperparameter complicato
- **Neural Networks:** capacità superiore su dati ad alta dimensionalità, ma "scatola nera"
- **Trade-off:** alberi per dataset piccoli-medi e interpretabilità richiesta; NN per big data e quando l'interpretabilità non è critica

6.10.5 Varianti Specializzate

6.10.5.1 Alberi Potati

Rimozione iterativa di nodi per ridurre overfitting:

- **Cost-Complexity Pruning:** elimina nodi che non riducono significativamente l'errore
- **Reduced Error Pruning:** rimuove nodi usando un validation set

6.10.5.2 Extra Trees (Extremely Randomized Trees)

Simile ai Decision Tree, ma sceglie split **casualmente** invece di deterministicamente. Veloce su dataset grandi, introduce varianza che riduce overfitting su alcuni dataset.

6.10.5.3 Conditional Inference Trees

Alberi che utilizzano test statistici per la selezione di split, meno biased verso feature con più livelli.

6.11 Prompt

Chapter 7

Random Forest

7.1 Modello

7.1.1 Logica dell'Algoritmo

Un **Random Forest** è un **ensemble method** (metodo di insieme) che combina **molteplici decision trees** per ottenere una previsione finale migliore rispetto a un singolo albero.

Idea fondamentale: ridurre l'overfitting e l'instabilità degli alberi singoli tramite:

1. **Bootstrap Aggregating (Bagging):** addestrare ogni albero su un campione bootstrap diverso dei dati

2. **Feature Randomness:** ad ogni split, considerare solo un subset casuale di feature
3. **Averaging/Voting:** combinare le previsioni di tutti gli alberi (media per regressione, majority voting per classificazione)

7.1.2 Procedura di Training

1. **Per ogni albero t da 1 a T (numero di alberi, solitamente 100-1000):**
 - a. Creare un **dataset bootstrap** D_t campionando n istanze **con reinserimento** dai dati originali
 - b. Addestrare un **decision tree** su D_t con i seguenti vincoli:
 - Ad ogni nodo, considerare solo $m_{\text{try}} = \sqrt{p}$ feature (per classificazione) o $m_{\text{try}} = p/3$ (per regressione)
 - Crescere l'albero **completamente** (senza potatura) fino a purezza (overfitting è OK, verrà mitigato dal bagging)
2. **Per la previsione:**
 - **Classificazione:** far votare tutti gli alberi $\rightarrow$ classe finale = moda (più frequente)
 - **Regressione:** far predire tutti gli alberi $\rightarrow$ valore finale = media

7.1.3 Motivazione Teorica

Perché il Bagging funziona?

Se abbiamo un modello con varianza alta (come un albero singolo), combinare molti modelli indipendenti (addestrati su dataset diversi) **riduce la varianza**:

$$\text{Var}(\bar{X}) = \frac{\text{Var}(X)}{N}$$

Se gli alberi fossero completamente indipendenti, la varianza della media calerebbe di un fattore di T (numero di alberi). In pratica non sono indipendenti (correlati perché addestrati sugli stessi dati), quindi la riduzione è minore, ma comunque significativa.

Perché la Randomness nelle Feature?

Il termine m_{try} introduce **de-correlazione** tra gli alberi:

- Se usassero tutte le feature, tutti gli alberi farebbero gli stessi split iniziali (molto correlati)
- Considerando solo $\sqrt{p}$ feature casuali, alberi diversi imparano dipendenze diverse $\rightarrow$ sono meno correlati
- Alberi meno correlati $\rightarrow$ riduzione della varianza nel voting

7.2 Complessità Computazionale

7.2.1 Training

$$O(T \cdot n \log(n) \cdot p)$$

Dove:

- T = numero di alberi (solitamente 100-1000, quindi fattore significativo)
- n = numero di osservazioni
- $\log(n)$ = profondità media di un albero bilanciato (senza potatura)
- p = numero di feature (ma con feature randomness, solo $\sqrt{p}$ considerate)

Spiegazione:

- Per ogni albero (T iterazioni):
 - Creare bootstrap: $O(n)$
 - Trainare albero: $O(n \log(n) \cdot m_{\text{try}})$ dove $m_{\text{try}} = \sqrt{p} \rightarrow O(n \log(n) \sqrt{p})$
- **Total:** $O(T \cdot n \log(n) \sqrt{p})$

7.2.2 Parallelizzazione

Random Forest è **intrinsecamente parallelizzabile**:

- Ogni albero è indipendente $\rightarrow$ addestrare su processori diversi
- Con K processori, speedup $\approx K$ (idealmente)
- Pratica standard: usare tutti i core disponibili

7.2.3 Inference (Previsione)

$$O(T \cdot d)$$

Dove d è la **profondità media di un albero**.

Per alberi non potati, $d \approx \log(n)$, quindi:

$$O(T \log(n))$$

Confronto con singolo albero: $O(\log(n))$ vs $O(T \log(n))$ — inference è più lenta di un albero singolo, ma comunemente accettabile.

7.2.4 Memoria

$$O(T \cdot n \cdot d)$$

Memorizzare T alberi, ognuno con $O(n)$ nodi nel peggiore (completo):

Per alberi bilanciati: $O(T \cdot n)$ (lineare in T).

7.2.5 Scalabilità: Conclusioni

Vantaggi:

- Training scala bene con n (logaritmico) e p (sqrt)
- Parallelizzazione è naturale e efficiente
- Può gestire dataset molto grandi (milioni di istanze)

Svantaggi:

- fattore T (numero di alberi) aggiunge overhead
- Memoria cresce linearmente con T
- Per interpretabilità, T grande rende difficile ispezionare

Pratica: Random Forest è considerato uno dei migliori compromessi tra performance e scalabilità, secondo solo ai metodi gradient-based (Gradient Boosting, LightGBM) per dataset molto grandi.

7.3 Rappresentazione Interna

7.3.1 Struttura del Modello

Un Random Forest è rappresentato come:

Forest = [Tree_1, Tree_2, ..., Tree_T]

Dove ogni Tree_i è un Decision Tree completo (no pruning)

Internamente:

- **T alberi indipendenti**, ognuno con una struttura ricorsiva (nodo $\rightarrow$ split $\rightarrow$ figli)
- **Ogni albero ha:** feature indice, threshold, puntatori ai figli, predizioni nelle foglie

7.3.2 Implicazioni per la Spiegabilità

Contro:

- **Molto opaco globalmente:** con $T = 100-500$ alberi, è impossibile ispezionare manualmente il modello completo
- **Difficile tracciare ragionamento:** non puoi seguire una singola catena di decisioni (ci sono 100 catene parallele)
- **Aggregazione nascosta:** il voting/averaging che determina la previsione finale non è facilmente interpretabile

Pro:

- **Interpretabile localmente:** puoi estrarre il singolo albero più importante (quello che spiega meglio una previsione)
- **Feature importance naturale:** misurata dalla riduzione di impurità media su tutti gli alberi
- **Out-of-Bag (OOB) error:** stima di generalizzazione gratuita, senza bisogno di validation set separato

Confronto con altri modelli:

- **DT singolo:** interpretabile globalmente, ma instabile
- **Random Forest:** opaco globalmente, ma stabile; interpretabilità per feature
- **SVM:** completamente opaco, nessuna feature importance naturale

7.4 Vincoli sui Dati

7.4.1 Nessuna Assunzione Strutturale

Come Decision Trees singoli, Random Forest **non ha assunzioni** su:

- Linearità
- Normalità
- Omoschedasticità
- Indipendenza (relativa)

7.4.2 Bilanciamento delle Classi

Random Forest eredita il problema dei Decision Trees per **classi sbilanciate**:

- Il voting del majority può favorire la classe maggioritaria
- Su dataset 95% negativi / 5% positivi, il modello potrebbe predire sempre negativo

Soluzioni:

- **Class weights:** pesi inversi alla frequenza per ogni albero
- **Stratified bootstrap:** assicurare che ogni bootstrap mantenga la proporzione di classi
- **Threshold tuning:** non usare 50% come confine, usare una soglia ottimale

7.4.3 Feature di Tipi Diversi

Random Forest gestisce **feature categoriche e numeriche** naturalmente:

- Non richiede encoding (come LR, LogR)
- Split su feature categoriche sono gestiti automaticamente
- Nessun vincolo di scala (a differenza di SVM)

7.4.4 Nessun Vincolo sulla Multicollinearità

A differenza di LR/LogR, Random Forest **non è sensibile a multicollinearità**:

- Se due feature sono correlate, l'albero semplicemente ne usa una
- Non causa instabilità come nei modelli lineari

7.5 Capacità Predittive

7.5.1 Punti di Forza

1. Eccellente Performance su Dati Reali

- Spesso outperform molti altri algoritmi su dataset variegati
- Cattura pattern non lineari e interazioni naturalmente

2. Robusto a Outlier e Rumore

- Gli outlier influenzano alcuni alberi, non tutti
- Voting/averaging mitiga l'impatto

3. Gestisce Pattern Complessi

- Interazioni tra feature
- Feature categoriche e numeriche miste
- Relazioni non monotone

4. Riduce Overfitting dei Singoli Alberi

- Bagging + feature randomness mitiga fortemente l'instabilità
- Più stabile di un singolo albero

5. Feature Importance Nativa

- Misura diretta dell'importanza di ogni feature
- Utile per feature selection e interpretazione

6. Scalabilità

- Parallelizzabile
- Gestisce dataset grandi

7.5.2 Punti di Debolezza

1. Interpretabilità Scarsa Globalmente

- Con 100-500 alberi, è impossibile ispezionare il modello
- Black box dal punto di vista dell'interpretazione globale

2. Sensibilità a Feature Dominanti

- Se una feature ha molte categorie, l'albero può over-splitarvi
- Bias verso feature ad alta cardinality (come DT singolo)

3. Memoria e Tempo di Inference

- Più lento di un singolo albero o di modelli lineari
- Memoria proporzionale a $T \times \text{dimensioni albero}$

4. Hyperparameter Tuning

- Parametri principali: T (numero alberi), max_depth , min_samples_split , m_try
- Tuning male = performance ridotta

5. Predizione Non Probabilistica per Classificazione

- Output è probabilità (frazione di alberi che votano classe) — non calibrata come LogR

7.6 Metriche per la Confidenza

7.6.1 Metriche di Classificazione Standard

Stesse metriche di altri classificatori:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN) / (TP + TN + FP + FN)$

- **Precision:** $TP/(TP + FP)$
- **Recall/Sensitivity:** $TP/(TP + FN)$
- **Specificity:** $TN/(TN + FP)$
- **F1-Score:** media armonica di Precision e Recall
- **ROC Curve e AUC:** trade-off tra TPR e FPR

7.6.2 Probabilità di Classe (Voting)

Random Forest produce naturalmente **probabilità della classe** per classificazione:

$$P(y = k) = \frac{\text{numero di alberi che votano classe } k}{T}$$

Interpretazione:

- Se 80 su 100 alberi votano "Positivo", $P(y = 1) = 0.80$
- Questa è una **stima di confidenza** della previsione

Caveat: queste probabilità non sono ben calibrate (non riflettono accuratezza vera come in LogR). Per probabilità calibrate, usare Platt scaling o isotonic regression post-hoc.

7.6.3 Out-of-Bag (OOB) Error

Misura di validazione **gratuita** senza validation set separato:

Per ogni istanza di training, gli alberi addestrati su bootstrap che **non contengono** quell'istanza (out-of-bag) la predicono. OOB error = errore medio su queste predizioni.

$$\text{OOB Error} = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\text{predizione OOB}_i \neq y_i]$$

Vantaggi:

- Stima di generalizzazione senza bisogno di validation set
- Correlato con test error (anche se non identico)
- Utile per tuning di hyperparameter

7.6.4 Metriche di Regressione

Per regressione:

- **Mean Squared Error (MSE):** $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$
- **Root Mean Squared Error (RMSE):** $\sqrt{MSE}$
- **Mean Absolute Error (MAE):** $\frac{1}{n} \sum |y_i - \hat{y}_i|$
- **R²:** frazione di varianza spiegata
- **OOB RMSE:** alternativa a cross-validation

7.7 Metriche per la Comprensione e Spiegabilità

7.7.1 1. Feature Importance (Impurità Media)

La metrica più importante di Random Forest per l'interpretazione:

$$\text{Importance}_j = \frac{1}{T} \sum_{t=1}^T \sum_{\text{nodi dove feature } j \text{ fa split}} \text{IG}_{t, \text{nodo}}$$

Dove IG è l'**information gain** (riduzione di impurità).

Interpretazione:

- Feature con importanza alta contribuiscono molto alle decisioni
- Ordinare per importanza identifica le feature più rilevanti
- Utile per feature selection e comunicazione ai non-esperti

Visualizzazione: bar plot di feature ordinate per importanza.

7.7.2 2. Feature Importance (Permutation-Based)

Alternativa al metodo di impurità:

- Permutare casualmente una feature nei dati di test
- Misurare il degrado in performance
- Grandi degni = feature importante

Vantaggio: meno biased verso feature ad alta cardinality (come il metodo di impurità).

7.7.3 3. Partial Dependence Plot (PDP)

Mostra come la previsione media varia al variare di una feature:

$$\text{PDP}_j(x_j) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_j, x_{-j}^{(i)})$$

Dove $x_{-j}^{(i)}$ sono i valori di altre feature dall'istanza i , mantenendo x_j fisso.

Visualizzazione: linea o curva che mostra l'effetto della feature sulla previsione.

Caveat: se la feature è correlata con altre, l'effetto può essere spurio.

7.7.4 4. Individual Conditional Expectation (ICE) Plot

Versione per-istanza di PDP:

- Mostrare come la previsione cambia al variare di una feature per singoli campioni
- Linee per campione (invece che media su tutti)

Utilità: capire se l'effetto di una feature è uniforme o varia tra istanze.

7.7.5 5. SHAP (SHapley Additive exPlanations)

Metodo avanzato per spiegabilità:

- Assegnare ogni output tra le feature usando valori Shapley dalla teoria dei giochi

- Applicabile a qualsiasi modello
- Produce spiegazioni teoricamente fondate

Vantaggio: considerare interazioni e correlazioni tra feature.

7.7.6 6. Single Tree Extraction

Per una previsione specifica:

- Estrarre il singolo albero che ha la maggiore "responsabilità" della previsione
- Mostrare il cammino dalla radice alla foglia di quell'albero
- Comunicare il ragionamento tramite quello specifico albero

Limite: può essere fuorviante perché ignora il contributo degli altri alberi.

7.8 Limiti di Predizione

7.8.1 Sensibilità a Feature Dominanti

Come Decision Trees, Random Forest tende a preferire feature con **molte categorie o valori unici**:

- Feature categoriche con 100 valori vs feature numerica continua
- Questo bias persiste anche con feature randomness

Mitigazione: usare permutation-based feature importance, che è meno biased.

7.8.2 Difficoltà con Dati Molto Sbilanciati

Se le classi sono estremamente sbilanciate (99% vs 1%):

- Il majority voting favorisce la classe maggioritaria
- Accuratezza può essere alta ma recall sulla classe rara bassa

Mitigazione: usare class weights, stratified bootstrap, o threshold tuning.

7.8.3 Scarsa Extrapolazione

Random Forest è un **interpolatore locale**:

- Non può predire valori significativamente fuori dal range di training
- Per regressione, le predizioni rimangono nel range dei dati di training
- Non adatto per trend/trend extrapolation

7.8.4 Performance su Feature Numeriche Pure

Su dataset con solo feature numeriche e pattern lineare, modelli lineari (LR) superano spesso Random Forest:

- RF "impara" la linearità tramite molti split ortogonali → inefficiente
- LR cattura la linearità con pochi parametri

7.8.5 Sensibilità a Iperparametri

Sebbene meno sensibile di SVM, Random Forest ha iperparametri critici:

- **T (numero alberi):** troppo pochi = underfitting; non ha limite superiore strict
- **max_depth:** alberi troppo profondi = overfitting; troppo poco profondi = underfitting
- **min_samples_split:** controlla quando smettere di splittare
- **m_try (feature per split):** influenza la diversità tra alberi

7.9 Limiti di Spiegabilità

7.9.1 Opacità Globale

Con $T = 100-500$ alberi:

- **Impossibile ispezionare manualmente** il modello completo
- Non puoi tracciare una singola sequenza di decisioni (ce ne sono T parallele)
- Il modello rimane una "**scatola nera**" per la maggior parte degli utenti

Contrasto con DT singolo: un albero piccolo è completamente trasparente; un RF non lo è.

7.9.2 Interpretazione di Feature Importance Ambigua

Feature importance (impurità) conta quante volte una feature fa split, non necessariamente quanto sia causale:

- Una feature correlata ad un'altra può avere bassa importanza se l'altra è splittata per prima
- Importanza non implica causalità

Soluzione: usare permutation-based importance (meno biased) o SHAP.

7.9.3 Interazioni Non Esplicite

Random Forest cattura interazioni automaticamente (feature A influenza come feature B splitta), ma:

- Le interazioni rimangono implicite nella struttura dell'albero
- Difficile comunicare "feature A e B interagiscono così"

Soluzione: usare SHAP per analizzare interazioni specifiche.

7.9.4 Dipendenza da Iperparametri

La spiegazione fornita da feature importance dipende da:

- max_depth (alberi più profondi = più split su feature rare)
- m_try (feature randomness influenza quali feature sono considerate)
- T (più alberi = più stabile, meno varianza)

Se si cambia iperparametri, feature importance cambia → spiegazione non è robusta.

7.9.5 Difficoltà di Spiegazione per Non-Esperti

Comunicare il risultato di RF a un non-esperto richiede:

- Tradurre "feature importance" in linguaggio intuitivo
- Evitare di mostrare 100 alberi
- Usare visualizzazioni (SHAP, PDP) per semplificare

7.10 Confronto con altri Algoritmi

7.10.1 Vs. Decision Tree Singolo

Aspetto	DT Singolo	Random Forest
Interpretabilità	★★★★★ Altissima	★★ Bassa
Overfitting	★★★★★ (alto)	★ (basso, mitigato)
Stabilità	★ (instabile)	★★★★★ (stabile)
Performance	★★★ Media	★★★★★ Eccellente
Scalabilità	★★★★ Buona	★★★★ Buona (parallelizzabile)
Quando usare	Max interpretabilità	Quando performance conta

7.10.2 Vs. Logistic Regression

Aspetto	LogR	Random Forest
Interpretabilità	★★★★ Alta	★★ Bassa
Performance Dati Lineari	★★★★★ Eccellente	★★ Inefficiente
Performance Dati Non-Lineari	★ Scarsa	★★★★★ Eccellente
Feature Categorie	★ (richiede encoding)	★★★★★ Naturale
Quando usare	Pattern lineare + interpretabilità	Pattern complesso, features miste

7.10.3 Vs. SVM

Aspetto	SVM	Random Forest
Scalabilità	☆☆ Scarsa $O(n^3)$	☆☆☆☆ Buona
Performance	☆☆☆☆ Buona	☆☆☆☆ Spesso migliore
Interpretabilità	☆☆ Scarsa	☆☆ Scarsa
Hyperparameter Tuning	☆☆ Sensibile	☆☆☆ Meno sensibile
Feature Importance	✗ No naturale	✓ Sì, nativo
Quando usare	Dati piccoli/medi stabili	Dati grandi, performance critica

7.10.4 Vs. Gradient Boosting

Aspetto	Gradient Boosting	Random Forest
Performance	☆☆☆☆ Spesso migliore	☆☆☆☆ Buona
Velocità Training	☆☆ Lenta (sequenziale)	☆☆☆☆ Veloce (parallela)
Hyperparameter Tuning	☆ Difficile (molti parametri)	☆☆☆ Più semplice
Risk Overfitting	☆☆ Alto	☆ Basso
Quando usare	Massima performance (con tuning)	Performance buona, implementazione semplice

7.10.5 Vs. Neural Networks

Aspetto	Neural Networks	Random Forest
Capacità	★★★★★ Illimitata	★★★★★ Molto buona
Dati Necessari	★ Molto (milioni)	★★★★★ Meno (migliaia)
Feature Importance	✗ Molto difficile	✓ Nativa
Interpretabilità	★ Nessuna	★★ Poca
Training	★★ Lungo, GPU-friendly	★★★ Veloce, CPU
Quando usare	Big data, no interpretabilità	Medium data, need feature importance

7.11 Varianti e Estensioni

7.11.1 1. Extra Trees (Extremely Randomized Trees)

Simile a Random Forest, ma con maggiore randomness:

- Split threshold è **casuale** (non cerca il miglior threshold)
- Meno computazionalmente costoso (no search del threshold)
- Spesso produce alberi meno profondi e più generali
- Trade-off: performance leggermente minore, ma velocità molto maggiore

7.11.2 2. Gradient Boosting (XGBoost, LightGBM, CatBoost)

Evoluzione sequenziale di Random Forest:

- Anziché addestrare alberi in parallelo (indipendenti), li addestra in **sequenza**
- Ogni albero corregge gli errori del precedente
- Spesso outperform RF, ma richiede tuning più accurato
- Più lento per training ma spesso migliore in performance

7.11.3 3. Isolation Forest

Specializzato per **anomaly detection**:

- Alberi costruiti per isolare anomalie (non per classificazione regolare)
- Anomalie sono isolate in meno split rispetto a punti normali
- Veloce e efficiente per alta dimensionalità

7.11.4 4. Random Forest per Regressione

Applica la stessa logica ma:

- Predizione finale è **media** (non voting)
- Feature importance basata su riduzione MSE (non impurità)
- Output naturalmente continuo

7.12 Raccomandazioni Pratiche

7.12.1 Quando Usare Random Forest

✓ Ottime scelte:

- Dataset di qualsiasi taglia (piccolo a medio)
- Pattern non lineari e complessi
- Necessità di feature importance
- Features miste (categoriche e numeriche)
- Quando interpretabilità non è critica ma performance sì

✗ Cattive scelte:

- Massima interpretabilità richiesta (es. medicina regolata, finanza)
- Dati puramente lineari (LR è più efficiente)
- Inferenza in real-time su devices con risorse limitate (T alberi è lento)
- Quando è critico capire il "perché" di ogni singola previsione

7.12.2 Hyperparameter Tuning Consigliato

1. Numero di alberi (**n_estimators**):

- Iniziare con 100-200
- Aumentare fino a quando OOB error non migliora
- Diminishing returns dopo 500-1000

2. Profondità massima (**max_depth**):

- Iniziare senza limite (None), poi ridurre se overfitting
- Tipicamente: 10-20 per dataset di taglia media

3. Min samples per split (**min_samples_split**):

- Default: 2 (troppo aggressivo)
- Provare: 5, 10, 20
- Aumentare per ridurre overfitting

4. Feature per split (**max_features**):

- Classification: $\sqrt{p}$ (default, di solito ottimale)
- Regressione: $p/3$ (default)
- Provare: 'auto', 'sqrt', 'log2'

5. Class weights (se classi sbilanciate):

- 'balanced' per pesi automatici (inversi alla frequenza)

7.12.3 Miglior Pratica di Validazione

1. Dividere in train (70-80%), test (20-30%)
2. Training su train set con OOB error per validazione interna
3. Validare su test set separato
4. Cross-validation (5-10 fold) per più stabilità

7.12.4 Feature Engineering

Random Forest è **resistente a feature engineering scadente**:

- Non richiede scaling
- Gestisce feature categoriche automaticamente
- Non sensibile a feature correlate

Quindi focus su:

- Feature relevance (sono importanti per il problema?)
- Feature nuove (es. interazioni derivate manualmente) se performance non è sufficiente

7.13 Prompt

Chapter 8

XGBoost: Extreme Gradient Boosting

8.1 Modello

8.1.1 Logica dell'Algoritmo

XGBoost (Extreme Gradient Boosting) è un **ensemble method sequenziale** che costruisce alberi decisionali uno dopo l'altro, dove ogni nuovo albero **corregge gli errori del precedente**. A differenza di **Random Forest** (alberi paralleli indipendenti), XGBoost è un **boosting algorithm** che combina predizioni tramite:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Dove:

- $\hat{y}_i^{(t-1)}$ è la previsione cumulativa dopo t-1 alberi

- $f_t(x_i)$ è il nuovo albero che predice i residui dell'iterazione precedente

8.1.2 Obiettivo Regularizzato

XGBoost minimizza un obiettivo di loss regularizzato:

$$L(\phi) = \sum_{i=1}^n l(\hat{y}_i, y_i) + \sum_{k=1}^K \Omega(f_k)$$

Dove:

- Primo termine: **loss function** (errore tra previsione e realtà)
- Secondo termine: **regularizzazione** che penalizza la complessità dell'insieme di alberi

La regularizzazione è:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2$$

Dove:

- γT penalizza il numero di foglie (T)
- $\frac{1}{2} \lambda \sum_{j=1}^J w_j^2$ penalizza la magnitudine dei pesi delle foglie

Interpretazione: il termine di regularizzazione preferisce alberi semplici e pesi piccoli, **riducendo overfitting**.

8.1.3 Gradient Tree Boosting (Secondo Ordine)

Aniché minimizzare direttamente, XGBoost usa un **secondo-ordine approximation** della loss:

$$L^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Dove:

- $g_i = \frac{\partial l}{\partial \hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$ è il **primo-ordine gradiente** (come in GBM standard)
- $h_i = \frac{\partial^2}{\partial \hat{y}^{(t-1)^2}} l(y_i, \hat{y}^{(t-1)})$ è il **secondo-ordine gradiente** (novità di XGBoost)

L'uso del secondo-ordine consente un'**ottimizzazione più accurata** rispetto a metodi che usano solo il primo-ordine.

8.1.4 Scoring Function per Split

Per una struttura d'albero fissa, il **quality score** è:

$$\text{Score}(q) = -\frac{1}{2} \sum_{j=1}^T \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Dove:

- I_j è l'insieme di istanze nella foglia j
- Questo score misura quanto una struttura d'albero è "buona"

- Valori negativi più grandi (più negativi) = alberi migliori

8.2 Complessità Computazionale

8.2.1 Training (Esatto)

$$O(K \cdot d \cdot n \log(n))$$

Dove:

- K = numero di alberi (iterazioni)
- d = profondità massima dell'albero
- n = numero di osservazioni
- $\log(n)$ = sorting dei dati per trovare gli split

Per ogni albero, l'algoritmo "exact greedy":

- Ordina i dati per feature: $O(n \log n)$ once per feature
- Enumera split con accumulo di gradienti: $O(n)$

8.2.2 Training (Approssimato - con Quantile Sketch)

$$O(K \cdot d \cdot n)$$

Usando il **weighted quantile sketch** (novità di XGBoost):

- Propone candidate split points in modo intelligent
- Non richiede sorting completo
- Ottimale per dataset out-of-core

8.2.3 Inference (Previsione)

$$O(K \cdot d)$$

Dove K è il numero di alberi, d la profondità media:

- Attraversare K alberi sequenzialmente
- Per ogni albero, seguire il cammino dalla radice alla foglia (profondità d)

8.2.4 Memoria

$$O(K \cdot n \cdot d + \text{block structure})$$

- Memorizzare K alberi: $O(K \cdot n)$ nel peggiore (tutte le istanze in ogni albero)
- Block structure per parallelizzazione: $O(n)$ aggiuntivo

8.2.5 Scalabilità: Conclusioni

Vantaggi rispetto a Random Forest:

- **Sequenziale è più efficiente:** ogni albero è costruito per correggere errori specifici

- **Alberi più piccoli:** con boosting, alberi più shallow (tipicamente profondità 5-8) vs RF (profondità $\leq n$)
- **Meno alberi necessari:** 100-500 alberi vs RF 1000+
- **Out-of-core:** XGBoost supporta nativamente dati che non fit in memoria

Svantaggi:

- **Sequenziale:** training è sequenziale, non parallelizzabile come RF
- **Sensibile ai parametri:** il tuning di learning rate, depth, regularization è critico

Trade-off: XGBoost è ~10x più veloce di Random Forest per item su dataset moderati (Chen & Guestrin, 2016).

8.3 Rappresentazione Interna

8.3.1 Struttura del Modello

XGBoost rappresenta internamente il modello come:

Ensemble = [Tree_1, Tree_2, ..., Tree_K]

Previsione = $\sum_k f_k(x)$

Dove ogni Tree_k è:

- Struttura: split feature, threshold, default direction
- Pesì: predizioni per ogni foglia
- Gradienti: accumulati durante training

8.3.2 Differenze da Random Forest

Aspetto	Random Forest	XGBoost
Alberi	Indipendenti, paralleli	Sequenziali, dipendenti
Costruzione	Campioni casuali (bootstrap)	Su residui dell'albero precedente
Profondità	Spesso profonde (10-20)	Solitamente basse (3-8)
Numero	Tanti (500-2000)	Meno (50-500)
Pesi	Voting majority	Somma pesata

8.3.3 Implicazioni per la Spiegabilità

Contro:

- Ancora più opaca di RF (500 alberi è molto)
- L'ordine degli alberi importa (sequenziale vs parallelo)

- Difficile tracciare quale albero ha causato una previsione

Pro:

- Feature importance è più **stabile** (alberi correlati meno che in RF)
- SHAP (sviluppato dagli stessi autori) è ottimizzato per XGBoost
- Ogni albero corrisponde a una "correzione" specifica di errori

8.4 Vincoli sui Dati

8.4.1 Nessuna Assunzione Strutturale

Come Random Forest e Decision Trees, XGBoost **non assume**:

- Linearità
- Normalità
- Omoschedasticità

8.4.2 Bilanciamento delle Classi

XGBoost è **più robusto** di DT singolo grazie alla regolarizzazione, ma può comunque soffrire di classi sbilanciate:

Soluzioni:

- **Scale_pos_weight**: parametro che pesa la classe minoritaria
- **Stratified k-fold**: durante cross-validation, mantenere proporzioni
- **Threshold tuning**: non usare 0.5, usare soglia ottimale

8.4.3 Normalizzazione delle Feature

A differenza di SVM, XGBoost **non richiede** normalizzazione:

- Usa split basati su percentili (invariante a scala)
- Gestisce naturalmente feature con range diversi

8.4.4 Gestione di Dati Sparsi

Novità importante di XGBoost: algoritmo sparsity-aware

Quando una feature ha valore mancante:

- L'istanza viene instradata ad una **default direction** (sinistra o destra)
- La default direction è **imparata dai dati**, non fissa
- Computazione è **lineare nel numero di non-missing entries** (50x più veloce su dati sparsi)

8.4.5 Dati Ponderati

XGBoost supporta **instance weights** tramite **weighted quantile sketch**:

- Calcola percentili ponderati (non solo uniforme)

- Teoricamente garantito a mantenere accuracy nei candidate splits

8.5 Capacità Predittive

8.5.1 Punti di Forza

1. Eccellente Performance Generale

- Ha vinto 17 di 29 Kaggle competitions nel 2015
- Spesso outperform altri metodi anche ensemble
- Performance quasi universalmente buona su dati reali

2. Flessibile per Problemi Diversi

- Classificazione binaria e multi-classe
- Regressione
- Learning to Rank
- Ranking object detection
- Supporta custom loss functions

3. Robusto a Dati Imperfetti

- Gestisce missing values
- Dati sparsi (one-hot encoding, etc.)
- Classe sbilanciate (con tuning)
- Outlier (soft-margin mitiga)

4. Pattern Non-Lineari e Interazioni

- Cattura relazioni complesse
- Interazioni automatiche tra feature
- Nessun feature engineering necessario (spesso)

5. Regolarizzazione Incorporated

- Riduce overfitting intrinsecamente
- Non serve validation set separato per early stopping
- Generalizza bene con pochi dati

8.5.2 Punti di Debolezza

1. Tuning Complesso

- Molti hyperparameter (learning rate, depth, lambda, gamma, etc.)
- Sensibile al tuning
- Default non sempre ottimali per il problema

2. Interpretabilità Scarsa

- 100-500 alberi è difficile da ispezionare
- Feature importance non è univoca (molti metodi differenti)

3. Non Probabilistico Nativo

- Output è predizione, non probabilità calibrate
- Per probabilità, usare sigmoid transformation

4. Sequenziale = Lento a Trainare

- Alberi dipendono dal precedente (non parallelizzabili)
- Più lento di RF puro per training

5. Curva di Apprendimento Steep

- Molti parametri da capire
- Documentazione densa
- Non intuitivo come Decision Tree o LR

8.6 Metriche per la Confidenza

8.6.1 Metriche Standard di Classificazione

Stesse metriche di altri classificatori:

- **Confusion Matrix:** TP, TN, FP, FN
- **Accuracy:** $(TP + TN)/(TP + TN + FP + FN)$
- **Precision, Recall, F1-Score:** standard
- **ROC Curve e AUC:** trade-off tra TPR e FPR

8.6.2 Margin/Distance Score

XGBoost produce un **margin score** (somma dei contributi degli alberi):

$$\text{margin}_i = \sum_{k=1}^K f_k(x_i)$$

Interpretazione:

- Valori più alti = più confidenza nella classe positiva
- Valori più bassi = più confidenza nella classe negativa
- Valori vicini a 0 = incertezza

8.6.3 Probabilità Calibrate (Sigmoid Transform)

Per ottenere probabilità da margin score:

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(-\text{margin})}$$

8.6.4 Early Stopping e Monitoring

XGBoost supporta **early stopping** usando metriche di validazione:

- Monitor AUC, log-loss, RMSE su validation set

- Stop quando metrica non migliora per N iterazioni
- Previene overfitting automaticamente

Vantaggio: non serve splitting train/val/test esplicito (uses internal validation).

8.7 Metriche per la Comprensione e Spiegabilità

8.7.1 1. Feature Importance (Gain-Based)

Misura la **riduzione di loss media** dovuta a ogni feature:

$$\text{Importance}_j = \frac{1}{K} \sum_{k=1}^K \text{Gain}_j^{(k)}$$

Dove $\text{Gain}_j^{(k)}$ è la riduzione di loss totale quando feature j fa split nell'albero k.

Interpretazione:

- Feature con gain alto sono "importanti" per il modello
- Somma a 100% (normalizzato)

Caveat: misure gain possono essere biased verso feature ad alta cardinality.

8.7.2 2. Feature Importance (Split-Based)

Conta quante volte una feature è stata usata per split:

$$\text{Cover}_j = \frac{1}{K} \sum_{k=1}^K \# \text{ splits su feature } j$$

Interpretazione: feature più frequenti sono considerate "importanti".

8.7.3 3. Feature Importance (SHAP)

SHAP (SHapley Additive exPlanations) è stato sviluppato specificamente per XGBoost:

- Assegna ogni predizione tra le feature usando valori Shapley
- Teoricamente fondata (teoria dei giochi)
- Fornisce spiegazioni sia globali che locali
- XGBoost ha ottimizzazioni native per SHAP

Utilizzo:

```
import shap
explainer = shap.TreeExplainer(xgboost_model)
shap_values = explainer.shap_values(data)
shap.summary_plot(shap_values, data)
```

8.7.4 4. Partial Dependence Plot (PDP)

Mostra come la previsione media cambia al variare di una feature:

$$\text{PDP}_j(x_j) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_j, x_{-j}^{(i)})$$

Visualizzazione: linea/curva che mostra relazione feature-output.

8.7.5 5. Individual Conditional Expectation (ICE)

Versione per-istanza di PDP:

- Linee separate per ogni campione
- Capire se l'effetto è uniforme o varia

8.7.6 6. Tree Visualization

Per problemi specifici, è possibile visualizzare alberi individuali:

```
import xgboost as xgb
xgb.plot_tree(model, num_trees=0) # Plot primo albero
```

Limite: con 500 alberi, visualizzare anche uno solo è complesso.

8.7.7 7. Explain Single Prediction (Contrastive)

Per una singola istanza, estrarre i contributi:

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_instance)
shap.force_plot(explainer.expected_value, shap_values, X_instance)
```

Questo mostra:

- Valore base (expected value)
- Contributi positivi (push verso +)
- Contributi negativi (push verso -)

8.8 Limiti di Predizione

8.8.1 Sensibilità al Tuning di Iperparametri

XGBoost ha **molte iperparametri critici**:

1. **learning_rate** (eta): shrinkage dopo ogni albero
 - Piccolo (0.01-0.1) = più stabile, training più lungo
 - Grande (0.3-1.0) = convergenza veloce, rischio overfitting
2. **max_depth**: profondità massima alberi
 - 3-5: shallow, underfitting
 - 10-15: typical, balanced
 - 20+: deep, overfitting

3. **min_child_weight**: istanze minime in foglia

- Alto = underfitting, modello semplice
- Basso = overfitting, modello complesso

4. **gamma**: perdita minima per split

- Alto = meno split (underfitting)
- Basso = più split (overfitting)

5. **lambda, alpha**: regolarizzazione L2, L1

- Alto = coefficienti piccoli (underfitting)
- Basso = coefficienti grandi (overfitting)

Problema: combinazioni non-lineari. Tuning è un'arte, non scienza.

8.8.2 Difficoltà su Dati Lineari

Se la vera relazione è lineare e semplice:

- LR è più efficiente (pochi parametri)
- XGBoost "forza" una soluzione tree-based (molti alberi per una retta)

8.8.3 Scarsa Generalizzazione con Molto Rumore

Se il dataset ha very high noise-to-signal ratio:

- XGBoost potrebbe overfitting nonostante regolarizzazione
- Mitigare con higher lambda, lower learning rate

8.8.4 Extrapolazione Debole

Come Random Forest, XGBoost **non** **extrapola**:

- Predizioni rimangono nel range osservato
- Non adatto per forecast trends

8.9 Limiti di Spiegabilità

8.9.1 Opacità della Struttura Sequenziale

Il fatto che ogni albero dipende dal precedente rende:

- Non chiaro quale albero "causa" una previsione
- Difficile tracciare il ragionamento passo per passo
- Un albero da solo potrebbe dare previsione completamente diversa

8.9.2 Interpretazione di Feature Importance Ambigua

XGBoost ha **tre metodi diversi** per feature importance (Gain, Cover, Frequency):

- Possono dare ranking **completamente diversi**
- Quale è "giusto"? Dipende dal caso d'uso

- Nessuno è intrinsecamente "corretto"

8.9.3 Difficoltà di Comunicazione

Spiegare una previsione di XGBoost a non-esperti è difficile:

- "Il modello è fatto di 200 alberi che si sommano"
- Non è intuitiva come "se Feature A > X allora Y" (DT)
- Non è semplice come "il coefficiente di A è 0.5" (LR)

8.9.4 SHAP è Computazionalmente Costoso

SHAP è il metodo migliore per spiegare XGBoost, ma:

- Calcolo è $O(K \cdot 2^p)$ nel peggiore (exponential in # features)
- Per 100+ features, diventa prohibitively slow
- TreeSHAP (ottimizzato) è più veloce ma comunque caro

8.9.5 Instabilità di Feature Importance

Se parametri cambiano leggermente:

- Feature importance può cambiare drasticamente
- Spiegazioni non sono "robuste"

8.10 Confronto con altri Algoritmi

8.10.1 Vs. Random Forest

Aspetto	Random Forest	XGBoost
Velocità Training	Veloce (parallelo)	Lento (sequenziale)
Velocità Inference	Veloce	Veloce
Performance	★★★★ Buona	★★★★★ Eccellente
Overfitting	★★ Basso	★ Basso ma sensibile tuning
Interpretabilità	★★ Media	★★ Media (ma con SHAP)
Tuning	★★★ Semplice	★ Complesso
Quando usare	Velocità + buona performance	Massima performance

8.10.2 Vs. Logistic Regression

Aspetto	LogR	XGBoost
Interpretabilità	★★★★ Alta	★★ Bassa
Pattern Non-Lineare	★ Scarso	★★★★★ Eccellente
Tuning	★★★ Semplice	★ Complesso
Scalabilità	★★★★ Buona	★★★ Media
Dati Rari	★★★★ Buono	★★★ Buono (needs tuning)
Quando usare	Interpretabilità + baseline	Performance è prioritario

8.10.3 Vs. Neural Networks

Aspetto	Neural Networks	XGBoost
Capacità	★★★★★ Illimitata	★★★★ Molto buona
Dati Necessari	★ Molti (milioni)	★★★★ Meno (migliaia)
Training	★★ Lungo	★★★ Veloce
Interpretabilità	★ Nessuna	★★ SHAP disponibile
Quando usare	Big data + pattern visivo	Structured data + performance

8.10.4 Vs. LightGBM / CatBoost

LightGBM (Light Gradient Boosting Machine):

- Simile a XGBoost ma con **histogram-based split** finding
- ~2x più veloce di XGBoost
- Più basso memoria
- Meno maturo (comunità più piccola)

CatBoost (Categorical Boosting):

- Specializzato in dati categorici
- Gestisce nativamente categorie senza encoding
- Meno tuning necessario (default migliori)
- Performance leggermente superiore a XGBoost su dati categorici

Confronto:

Aspetto	XGBoost	LightGBM	CatBoost
Velocità	★★★★	★★★★★	★★★★
Performance	★★★★★	★★★★★	★★★★★
Tuning Facile	★★	★★★	★★★★
Dati Categorici	★★	★★★	★★★★★
Comunità	★★★★★	★★★★★	★★★
Stabilità	★★★★★	★★★★	★★★

8.11 Varianti e Estensioni

8.11.1 1. XGBoost Rank (Learning to Rank)

Specializzato per problemi di ranking:

- Ottimizza NDCG, MAP, ecc.
- Usato in search engines, ad ranking
- Gestisce query groups

8.11.2 2. XGBoost GPU

Supporto nativo per GPU NVIDIA (CUDA):

- ~10x più veloce di CPU
- Per dataset molto grandi
- Richiede GPU con memoria sufficiente

8.11.3 3. XGBoost per Distributed Training

- **Spark integration:** XGBoost su Spark cluster
- **Hadoop/MPI:** per distributed environments
- **Dask:** per parallel computing su CPU

8.11.4 4. Dart (Dropouts Meet Multiple Additive Regression Trees)

Variante di XGBoost che usa **dropout** (come neural networks):

- Droppa (scarta) alberi randomicamente durante training
- Riduce correlazione tra alberi
- Spesso migliore performance, specialmente su dataset piccoli

8.11.5 5. Linear Booster

Anziché tree boosting, usa **regressione lineare** a ogni step:

- Più semplice e interpretabile
- Più veloce
- Per pattern principalmente lineari

8.12 Raccomandazioni Pratiche

8.12.1 Quando Usare XGBoost

✓ Ottime scelte:

- Competizioni di ML (Kaggle, ecc.)
- Structured tabular data (no immagini/testo)
- Necessità di performance massima
- Dataset da migliaia a milioni di righe
- Mix di feature categoriche e numeriche
- Dati sparsi (one-hot encoding, ecc.)

✗ Cattive scelte:

- Testo/immagini (usare Deep Learning)
- Massima interpretabilità richiesta (usare LR o DT)
- Real-time inference su edge devices (slow inference)
- Dataset piccolissimi (< 1000 righe, rischio overfitting)
- Dati molto rumorosi (difficile tunare)

8.12.2 Workflow Consigliato

1. BASELINE: Train LR o DT semplice
↓
2. PARAMETER TUNING: Grid search / Random search / Bayesian
 - Iniziare con default: learning_rate=0.1, max_depth=6, lambda=1
 - Tuning in ordine: learning_rate → max_depth → lambda/alpha↓
3. VALIDATION: Usare 5-10 fold cross-validation
↓
4. MONITORING: Watch AUC/RMSE su validation set
 - Early stopping quando validation metric plateaus↓
5. INTERPRETATION: Usare SHAP per understand feature importance

8.12.3 Hyperparameter Tuning Dettagliato

8.12.3.1 Step 1: Learning Rate e Number of Rounds

```
# First: find good number of trees with default learning rate
xgb_params = {
```

```

    'eta': 0.1,
    'max_depth': 6,
    'min_child_weight': 1,
    'gamma': 0,
    'lambda': 1,
    'alpha': 0
}
# Train con early stopping, find best num_rounds

```

8.12.3.2 Step 2: Tree Depth e Min Child Weight

```

# Grid search over depth e min_child_weight
depths = [3, 5, 7, 9]
min_child_weights = [1, 3, 5, 7]
# Risultato: profondità tipicamente 5-8, min_child 1-3

```

8.12.3.3 Step 3: Regularizzazione (Lambda, Alpha)

```

# Una volta trovati depth e learning_rate
# Tuare lambda (L2) e alpha (L1)
lambdas = [0.1, 1, 10, 100]
alphas = [0, 0.1, 1, 10]

```

8.12.3.4 Step 4: Column Subsampling

```

# Ridurre numero di feature per split
# Aiuta con overfitting
colsample_bytree = [0.5, 0.7, 1.0]
colsample_bylevel = [0.5, 0.7, 1.0]

```

8.12.4 Feature Engineering Minimo Necessario

XGBoost è **robusto a feature engineering scadente**, ma alcuni trucchi aiutano:

1. **One-hot Encoding** per categoriche (XGBoost le gestisce, ma sparsity-aware accelera)
2. **Log Transform** per feature long-tailed (es. income)
3. **Interaction Features** (a volte aiuta): $x_1 \times x_2$
4. **Polynomial Features** (raramente necessario)

In generale: meno feature engineering, meglio è. Lasciar decidere a XGBoost.

8.12.5 Debugging Overfitting

Se il modello overfits (validation error cresce):

```

→ Aumentare lambda (L2 regolarizzazione)
→ Diminuire learning_rate
→ Ridurre max_depth
→ Aumentare min_child_weight
→ Aggiungere column/row subsampling

```

- Aumentare gamma
- Usare early stopping più aggressivo

8.12.6 Debugging Underfitting

Se il modello underfits (training error alto):

- Diminuire lambda
- Aumentare learning_rate
- Aumentare max_depth
- Diminuire min_child_weight
- Aumentare numero di rounds
- Disabilitare early stopping (train longer)

8.13 Considerazioni Etiche e Normative

8.13.1 GDPR Right to Explanation

XGBoost è un **black-box model** in contesti sensibili (credito, assicurazione):

- Necessaria spiegabilità per ogni decisione
- Usare SHAP per soddisfare requisiti GDPR
- Documentare feature importance

8.13.2 Fairness e Bias

Controllare per bias rispetto a protected attributes:

- Gender, Race, Age, etc.
- Usare fairness metrics (disparate impact ratio, etc.)
- Monitor SHAP values per protected groups

8.14 Prompt

8.15 ...

Chapter 9

Design and code implementation

Breve introduzione al capitolo

9.1 Tecnologie e strumenti

Di seguito viene data una panoramica delle tecnologie e strumenti utilizzati.

9.1.1 Tecnologia 1

Descrizione Tecnologia 1.

9.1.2 Tecnologia 2

Descrizione Tecnologia 2

9.2 Progettazione

9.3 Design Pattern utilizzati

9.4 Codifica

Chapter 10

Prompt Engineering

Breve introduzione al capitolo

Chapter 11

Conclusion

11.1 Consuntivo finale

11.2 Raggiungimento degli obiettivi

11.3 Conoscenze acquisite

11.4 Valutazione personale

Bibliography

- [1] Christoph Molnar, *Interpretable Machine Learning*. [Online]. Available: <https://christophm.github.io/interpretable-ml-book/>
- [2] Zizhao Hu, Mohammad Rostami, and Jesse Thomason, “Expert Personas Improve LLM Alignment but Damage Accuracy.”
- [3] “Which Table Format Do LLMs Understand Best?.” [Online]. Available: <https://www.improvingagents.com/blog/best-input-data-format-for-llms/>
- [4] Jason Wei *et al.*, “Chain of Thought Prompting Elicits Reasoning in Large Language Models.”
- [5] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa, “Large Language Models are Zero-Shot Reasoners.”
- [6] Taowen Wang *et al.*, “M²PT.”

Chapter 12

Glossary

categorical features – Categorical Features: Features in a dataset that represent discrete categories or groups, rather than continuous numerical values, and often require special handling in machine learning algorithms. [4](#)

classification – Classification: A type of machine learning problem in which the goal is to predict discrete class labels based on input features. [i](#), [2](#)

computational complexity – Computational Complexity: A measure of the amount of computational resources (time and space) that an algorithm requires to run, often expressed in terms of the size of the input data. [4](#)

XAI – Explainable Artificial Intelligence: A field of artificial intelligence that focuses on creating models and systems that can provide clear and understandable explanations for their decisions and actions. [i](#), [i](#), [2](#), [9](#)

LLM – Large Language Model: A type of artificial intelligence model that is trained on vast amounts of text data to understand and generate human-like language. [i](#), [i](#), [2](#), [7](#), [7](#), [7](#), [9](#), [12](#)

ML – Machine Learning: A subset of artificial intelligence that involves training algorithms to learn patterns from data and make predictions or decisions without being explicitly programmed. [i](#), [1](#)

MSE – Mean Squared Error: A common loss function used in regression problems that measures the average squared difference between predicted values and actual target values. [12](#)

notai – Artificial Intelligence: Field of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, problem-solving, and understanding natural language. [1](#)

objective function – Objective Function: A mathematical function that an algorithm optimizes during training, which defines the goal of the learning process and guides the model's adjustments to minimize error or maximize performance. [4](#)

PE – Prompt Engineering: The process of designing and optimizing prompts to effectively communicate with language models and elicit desired responses. [2](#)

regression – Regression: A type of machine learning problem in which the goal is to predict continuous numerical values based on input features. [i](#), [2](#)

SHAP – SHapley Additive exPlanations: A method for explaining the output of machine learning models by attributing the prediction to each feature. [i](#)